

XV6 RISC-V Book 中文版

来源：<https://github.com/shzhxh/xv6-riscv-book-CN>

README.md

原文：<https://github.com/shzhxh/xv6-riscv-book-CN/blob/master/README.md>

这是对xv6-riscv-book的宽松翻译。现在(2020-08-25)：

- 没有翻译最后一章：重新审视并发。
- 各章最后的**真实世界**和**练习**没有翻译。

还包含了一些原书没有的内容，有些分布在各章节的注释部分，有些独立成章：

- 9_磁盘的结构.md：介绍mkfs.c的代码

1_操作系统接口.md

原文：https://github.com/shzhxh/xv6-riscv-book-CN/blob/master/1_%E6%93%8D%E4%BD%9C%E7%B3%BB%E7%BB%9F%E6%8E%A5%E5%8F%A3.md

操作系统的任务是在多个进程之间共享计算机，并提供比硬件本身支持更有用的服务。操作系统抽象和管理底层的硬件，以使上层的程序不必关心硬件的细节。它也在多个进程之间共享硬件，这样多个进程就可以同时运行了。最终，操作系统为程序提供了受控的交互方式，让它们可以共享数据或同时工作。

操作系统使用接口来为用户程序提供服务。设计一个好的接口是困难的。一方面我们希望它简单以便于正确地实现，另一方面我们又希望能提供大量复杂的特性。解决这一矛盾的技巧是，在设计接口的时候让它们依赖一些机制，这些机制可以组合起来提供更多的通用性。

xv6实现了Unix的基本接口，并模仿了它的内部设计。

xv6有内核，用以为运行的程序提供服务。运行的程序，即进程，有自己的内存用以包含指令、数据和栈。

进程通过系统调用使用系统服务。这样，进程就在内核空间 and 用户空间之间切换。

在内核层面实现了进程隔离的机制。

本章的其余部分概述了xv6提供的服务：进程、内存、文件描述符、管道、文件系统，并讨论了shell。

shell只是一个普通的程序，它从用户那里读取命令并执行它们。它的代码在user/sh.c。

从内核态到用户态

在xv6启动的过程中，0号核在main函数里会执行userinit函数，标记initcode处的数据会复制到第一个用户进程的内存空间。标记initcode的数字是一段代码，它对应的内容在user/initcode.S，是要把文件init的内容放到内存里以用户态去执行。文件init编译自user/init.c。

user/init.c把文件sh放到内存里执行。文件sh编译自user/sh.c，把用户输入的命令调入内存中执行，它就是xv6的shell。shell就是不断地使用getcmd函数读取命令行的输入，然后使用runcmd函数来执行命令行的输入。

从用户态到内核态

系统调用怎么从用户态到的内核态呢？文件user/usys.pl生成user/usys.S，usys.S就是用来完成从用户态到内核态的代码。从代码可见，它是通过ecall产生一个异常来进入内核态。只要把usys.S生成的二进制文件链接进用户程序里，就可以使用系统调用了。

进程和内存

xv6的进程包含两部分：一是用户空间的内存(指令、数据和栈)，一是仅内核可见的进程状态。

相关系统调用的实现请看kernel/sysproc.c。

fork

文件位置：kernel/proc.c。作用：创建一个进程，新进程的内存和父进程是完全一样的。

fork函数一次调用两次返回，这很怪异，因为一般的调用都是只返回一次的。fork从父进程返回很正常，但为什么会从子进程返回呢？这是因为子进程继承了父进程的所有资源，这一句`*(np->tf) = *(p->tf)`相当于子进程复制了父进程运行的“快照”，子进程里也相当于进行了一个fork函数且从下一句开始执行。

对fork函数的一点分析

1. p为当前进程，np为子进程。
2. uvmcopy把父进程的内存复制给子进程。
3. np->tf->a0 = 0保证了子进程返回0，因为寄存器a0保存了函数的返回值。
4. return pid对于子进程来说，在汇编层面应该实际执行的返回寄存器a0的值。

exit

文件位置：kernel/proc.c。作用：让调用它的进程停止运行并释放资源(如内存、打开的文件等)。`exit`有一个整型的状态参数，通常0代表成功，1代表失败。

wait

文件位置：kernel/proc.c。作用：等待当前进程的某个子进程退出；如果有子进程退出，返回该进程的pid，并把退出状态复制到传递给它的地址上。如果不关心子进程的退出状态，可以直接给wait传递参数0。

exec

文件位置：kernel/exec.c。作用：把当前进程的内存替换为文件里保存的内存镜像并执行之。`exec`有两个参数，第一个是要执行的程序，

第二个这个程序的参数(以字符串数组的形式出现)。

`exec`首先用`namei`来打开文件`path`。然后，读取ELF header。xv6的程序是用ELF格式来读取的(ELF格式详见`kernel/elf.h`)。在一个ELF二进制里，ELF header(`struct elfhdr`)在头部的位置，接下来是些程序头(`struct proghdr`)，每个程序头都描述了一个必须被载入内存里的段(section)。xv6的程序只有一个程序头，但其它系统指令和数据可能会有不同的段(section)。

第一步是快速检查文件是否正确包含了一个ELF二进制。ELF二进制的开头部分是一个魔数，0x7f后面跟字符串"ELF"。如果魔数能对的上，`exec`就认为这个二进制是正确的。

`exec`使用`proc_pagetable`分配了一个没有映射的新页表，然后用`uvmalloc`来为每个ELF段分配内存，并用`loadseg`把每个ELF段载入内存。`loadseg`使用`walkaddr`来找到将要写入ELF段的物理地址，然后用`readi`来把文件中的内容载入到该地址。

可以用`objdump -p`来查看程序头的内容。在程序头里`filesz`可能比`memsz`小，这表示它们之间的间隙(gap)应该用0来填充，而不是从文件里读取。

接下来该分配和初始化用户的栈了。它只为栈分配了一个页。`exec`依次把参数复制到栈顶，然后把到这些参数的指针记录在`ustack`里。在传给`main`函数的`argv`列表里，最末尾放了一个空指针。`ustack`的前三个入口分别是假的返回PC，`argc`，`argv`的指针。

`exec`在栈的下面放了一个不可访问的页，这样栈溢出的时候就会报错。这个不可访问的页也使得`exec`可以处理太长的参数；当参数太长

的时候，`exec`用来把参数复制到栈的函数`copyout`将会提示目标页不可访问，并返回1。

在准备新的内存镜像的时候，如果`exec`检测到一个错误(比如无效的程序段)，将会跳转到标记`bad`，释放新的镜像，并返回1。`exec`必须等到系统调用执行成功才可以释放掉旧的镜像；如果旧的镜像没有了，系统调用就没有办法向它返回1了。`exec`的错误只可能发生在镜像创立期间。一旦镜像完成，`exec`就可以提交到新的页表并释放旧的页表。

`exec`按ELF文件定义的地址把ELF文件载入到内存。用户或进程可以在ELF文件里指定任意地址。这是危险的，因为这些地址可能指向内核，不管是有意还无意。一个不严谨的内核将导致若干不好的结果，从崩溃到对隔离机制的恶意破坏(如安全漏洞)。xv6进行了一系列检查来避免这些风险。如`if(ph.vaddr + ph.memsz < ph.vaddr)`检查两者相加是否溢出。在x86版的xv6中，如果`ph.memsz`足够大而溢出0x1000，用户是有可能把数据复制到内核里的。在RISC-V版的xv6中则无此问题，因为内核和用户分别有各自的页表。

内核开发者很容易忽略一个重要的检查，在真实的世界中由于没有进行检查而使用户程序获取内核权限的事情已经有很长的历史了。xv6对提供给内核的用户数据的检查很可能是不全面的，用户程序是有可能绕过xv6的隔离机制的。

sbrk

文件位置：kernel/sysproc.c。作用：为进程分配或回收内存。它有一个参数，代表要分配的字节数。它返回的是新分配内存的地址。

这个系统调用是通过`growproc`来实现的。`growproc`使用`uvmalloc`来分配内存，如果给的参数是正数。或使用`uvmdealloc`来释放内存，如果给

的参数是负数。

`uvmalloc` 首先使用 `kalloc` 来分配物理内存，然后再用 `mappages` 把 PTE 加到用户的页表里。`uvmdealloc` 调用 `uvmunmap` 实现其功能，`uvmunmap` 首先用 `walk` 来找到对应的 PTE，然后使用 `kfree` 来释放相应的物理内存。

xv6 里进程的页表不只是告诉硬件怎么映射到虚拟地址，也是分配给那个进程的物理内存页的唯一记录。这就是为什么 `uvmunmap` 在释放用户内存的时候需要对用户页表进行检查。

shell

文件位置：user/sh.c。shell 结构简单，在 `main` 函数里可以看到它的主循环就是不断地使用 `getcmd` 来读取用户输入。然后使用 `fork` 来创建一个子进程。父进程调用 `wait` 来等待子进程执行命令。子进程调用 `runcmd` 来执行真正的命令。

既然 `fork` 之后肯定要执行 `exec`，为什么不把它们合而为一呢？这是为了在 I/O 重定向的时候方便使用。`fork` 复制的内存基本上是无用的，会在随后被 `exec` 替换掉，为了防止复制过程的资源浪费，可以使用虚拟内存的技术(如写时复制 copy-on-write)。

在 xv6 里没有多用户的概念。用 Unix 术语来说，所有 xv6 进程都是作为根用户来运行的。

I/O 和文件描述符

一个文件描述符是一个小的整数，它代表了一个内核管理的对象(kernel-managed object)，进程对这个对象进行读写操作。进程可能通过如下方法获取文件描述符，打开文件、目录或设备，创建管道，复制一个已有的描述符。为简单起见，文件描述符指向的对象都被称

为是“文件”；文件描述符抽象出来文件、管道和设备的共同点，使它们看起来都像是字节流。

在每个进程表里都把文件描述符作为索引，这样每个进程都有文件描述符的私有空间，文件描述符都是从0开始计数。按照惯例，0是标准输入，1是标准输出，2是标准错误。shell就是使用的这个约定实现的I/O重定向和管道。shell始终打开了三个文件描述符，作为控制台的默认文件描述符。

相关系统调用的实现请看kernel/sysfile.c。

read

文件位置kernel/file.c。作用：从文件描述符fd所指向的文件读取n个字节，把它们复制进buf。返回值是读取到的字节数。每个文件描述符都记录了它在文件里的偏移位置，read就是从当前偏移位置开始读取数据的，然后在那个偏移量上递增读取的字节数。

write

文件位置kernel/file.c。作用：从buf写入n个字节到文件描述符fd所指向的文件。返回写入的字节数。它对文件描述符里偏移量的操作和read是一样的。

close

作用：释放一个文件描述符，这样这个文件描述符就可以被其它的系统调用(open、pipe、dup)使用了。总是优先分配当前进程里未使用的数字最小的描述符。

文件描述符和`fork`的交互可以比较容易地实现I/O重定向。`fork`把文件描述符表复制给了子进程，而`exec`在替换子进程的内存的时候并不会替换这个文件描述符表。这样shell在实现I/O重定向的时候，只要forking，重新打开关闭的文件描述符，执行新程序就可以了。

从中可见把`fork`和`exec`分开的好处：不用预先执行就可以修改子进程的文件描述符。

虽然`fork`复制了文件描述符表，但父子进程依然共享了底层文件的偏移。这个行为有助于顺序执行顺序输出的shell命令。

dup

复制一个已有的文件描述符，返回一个新的文件描述符，这个新的文件描述符和原文件描述符指向相同的底层I/O实体。(这个底层I/O实体可能是管道、索引结点或设备)。这两个文件描述符共用同一个位移，就像用`fork`复制的那样。

除了`fork`和`dup`，文件描述符之间是没有办法共享位移的。有了`dup`系统调用，我们就可以把标准输出复制给标准错误，进而在shell里实现`2>&1`这样的功能(把标准错误和标准输出都作为标准输出)。xv6不支持标准错误的I/O重定向，但你已经知道怎么实现它了。

文件描述符是个强有力的抽象，因为可以用同一个接口处理不同的实体：文件，设备和管道。

管道

管道用于进程间的通信，它其实是内核里一块小的缓冲区，这个缓冲区向进程提供了一对文件描述符，一个用于读而另一个用于写。向管道的一端写数据，会使管道的另一端可以读这些数据。

如下是程序wc的示例代码，它把标准输入连接到了一个管道的读取端。

```
int p[2];
char *argv[2];

argv[0] = "wc";
argv[1] = 0;

pipe(p);
if(fork() == 0) {
    close(0);
    dup(p[0]);
    close(p[0]);
    close(p[1]);
    exec("/bin/wc", argv);
} else {
    close(p[0]);
    write(p[1], "hello world\n", 12);
    close(p[1]);
}
```

程序调用`pipe`来创建管道，并把读和写的文件描述符记录在数组`p`里。执行`fork`后，父进程和子进程都有了指向管道的文件描述符。子进程把读取端复制给文件描述符0，关闭`p`里的文件描述符，执行`wc`。当`wc`从标准输入里读时，它就是从管道里读。父进程关闭管道的读取端，向管道写入数据，然后关闭写入端。

当管道里没有数据的时候，管道上的`read`要么等数据写入，要么等写入端的文件描述符被关闭；在后一种情况下，`read`将返回0，就像到达了数据文件的尾部一样。如果新数据不可能再到来，`read`就会一直阻塞，这就是在执行`wc`之前子进程要关闭管道的写端的原因之一：如果

有一个`wc`的文件描述符指向了管道的写端，`wc`就再也看不到文件结束了。

XV6 shell实现的管道和上面的代码是类似的，详见`user/sh.c`。子进程创建了一个管道。然后它为管道的左端和右端分别调用`fork`和`runcmd`，并且等待两边都完成。管道的右端可能也是一个包含了管道的命令，那么所有使用了管道的这些子进程就构成了一颗树。树的叶子是命令，而内部结点是等待左子结点和右子结点都完成的进程。理论上讲可以让内部结点从管道的左端开始执行，但这会增加复杂性。

看起来管道做的事，用临时文件重定向也可以做。但实际上，管道比临时文件至少有如下优势：

1. 管道会自动清除它自己，而使用重定向则要手动清除临时文件。
2. 管道可以传输任意长度的数据流，而重定向则要求磁盘有足够的空间来保存所有的数据。
3. 管道可以并行执行，而重定向只能依次执行。
4. 如果进行的是进程间通信，管道阻塞读和写这样的语义比文件的非阻塞语义更有效。

文件系统

XV6的文件系统提供了文件和目录，文件就是字节组成的数组，而目录由带着名字的文件和其它目录组成。

全部目录构成了一个树，它从`root`目录开始。**路径**给出了一个文件或目录的位置。路径除了可以从根目录开始，也可以从进程的**当前目录**开始，当前目录可以用`chdir`系统调用来改变。

可以创建文件或目录的系统调用：`mkdir`创建一个新的目录，`open`带着标志`O_CREATE`可以创建一个新的数据文件，`mknod`创建一个设备文件。

`mknod`只是创建一个没有任何内容的文件。它只是初始化这个文件的元数据，把文件类型设置为设备，设置`major`和`minor`以唯一对应一个内核设备。当一个进程要打开这个设备文件的时候，内核就不会把`read`和`write`系统调用传给文件系统了，而是传给内核设备。

`fstat`用于取出文件描述符所指向的那个文件的元信息，它会把这些信息填充到结构体`struct stat`里。

文件名不能代表文件本身，文件的底层实现是 **索引结点**，而一个索引结点可以有多个名字，叫做 **链接**。`link`系统调用可以给已有的索引结点再创建一个文件名。

对于同一个文件，不管用它的哪个文件名读写，其效果都是一样的。一个索引结点和它的 **索引结点号** 是一一对应的。

`unlink`系统调用用于从文件系统里移除一个文件名。当文件的链接记数为0且没有文件描述符指向它，那个文件的索引结点和磁盘空间才会被释放。

像`mkdir`，`ln`，`mr`这样的用于文件操作的shell命令，都被实现为用户态的程序。这样任何人都可以扩展用户程序了。这似乎看起来是显而易见的，但同时期的其它系统常常是把这些命令内建到shell里，然后又把shell内建到内核里。

但`cd`命令是个例外，它是内建在shell里的，因为它必须改变shell自身的当前工作目录。如果`cd`是一个常规命令，那么它就会作为子进程来

运行，它改变的是子进程的工作目录。而我们想改变的父进程的工作目录并没有改变。

2_xv6的组织.md

原文：https://github.com/shzhxh/xv6-riscv-book-CN/blob/master/2_xv6%E7%9A%84%E7%BB%84%E7%BB%87.md

操作系统有三个需求：多路复用、隔离和交互。本章概述了操作系统如何实现如上三个需求，本章还概述了xv6的进程。xv6-riscv在低层的实现很多是特定于riscv的，比如进程。

抽象物理资源

把物理资源抽象成系统调用接口，可以简化应用程序的编写，并使应用程序相互隔离，实践证明是个比较好的硬件抽象方法。

特权模式和系统调用

应用程序处于用户模式，以被强制隔离。内核处于管理模式，以执行特权指令。CPU从机器模式开始执行，主要是配置计算机。

应用程序通过ecall指令调用内核的功能。应用程序不应知道调用内核功能的内存地址，因为恶意程序可借此威胁系统安全。

内核的组织

操作系统的哪一部分放在管理模式下是设计的一个核心问题，宏内核的设计是把整个操作系统都放在内核里。这样的内核组织方式使得操作系统拥有完整的硬件权限，这有利于操作系统的不同部分间相互协同。但这也使操作系统的不同部分之间的接口变得复杂，从而容易出现错误。为了减少内核产生错误的风险，仅让少量的操作系统代码

运行在管理模式，而让大部分的操作系统都运行在用户模式，这样的内核组织方式叫微内核。

在微内核里，操作系统的大量功能都以服务的形式运行于用户态，其它程序以进程间通信的方式使用操作系统提供的服务。这样内核接口就仅包含少量的低级别的功能，诸如启动程序、发送消息、访问硬件等。

xv6的组织

xv6内核的源文件在kernel子目录下。以模块化的粗略概念组织在一起，各模块之间的接口定义在defs.h。

文件名	描述
bio.c	文件系统的硬盘块缓存
console.c	连接到用户的键盘和屏幕
entry.S	最初的启动指令
exec.c	exec系统调用
file.c	对文件描述符的支持
fs.c	文件系统
kalloc.c	物理页分配
kernelvec.S	管理来自内核的trap和计时器中断
log.c	文件系统日志和崩溃恢复

文件名	描述
main.c	在启动的时候控制其它模块的初始化
pipe.c	管道
plic.c	RISC-V的中断控制器
printf.c	格式化输出到控制台
proc.c	进程与调度
sleeplock.c	让出CPU的锁
spinlock.c	不让出CPU的锁
start.c	早期机器模式下的启动代码
string.c	C字符串和字节数组(byte-array)的库
swtch.S	线程切换
syscall.c	把系统调用对应到处理函数
sysfile.c	与文件相关的系统调用
sysproc.c	与进程相关的系统调用
trampoline.S	用户与内核切换的汇编代码
trap.c	管理traps和中断
uart.c	串口设备的驱动
virtio_disk.c	磁盘设备的驱动

文件名	描述
vm.c	管理页表和地址空间

进程概述

像其它Unix操作系统一样，xv6把进程作为隔离单元。内核实现进程所用到的机制包括用户模式\管理模式的标志(flag)，地址空间，线程的时间片。

进程的存在，使程序看上去拥有自己的CPU和内存。xv6使用页表来实现进程各自的地址空间。

每个进程都有自己的页表，这使得它们的物理内存互不重合。进程的虚拟地址空间从零开始，首先是指令，接下来是全局变量，然后是栈，最后是堆区(用于进程扩展内存)。最顶端的两个页是保留的，上面是trampoline，下面则映射的是切换到内核的trapframe。

xv6用`struct proc`记录每个进程的状态。重要的状态有页表，内核栈和运行状态。

每个进程都有一个运行着的线程来执行进程的指令。线程可以挂起和恢复。不同进程之间的切换，就是内核挂起当前运行着的线程并恢复其它进程的线程。每个进程都有两个栈：用户栈和内核栈。当进程执行用户指令的时候仅使用它的用户栈，它的内核栈是空的。当进程进入内核(因为系统调用或中断)，内核代码使用进程的内核栈，这时进程的用户栈还保存了数据只是不被使用了。内核栈是独立的，即使进程破坏了它的用户栈内核也可以继续执行。

进程使用ecall指令来做出一个系统调用。这个指令提升特权级并把程序计数器切换到内核定义的入口上。入口处的代码做的事就是切换到内核栈并执行对应的系统调用实现。当系统调用完成后，内核切换回用户栈并通过sret指令返回用户空间。

第一个进程

加电后先完成初始化，然后运行ROM里的bootloader，bootloader再把xv6内核载入内存。然后从xv6内核的`_entry`开始执行。内核被加载到0x80000000处，之所以不加载到0x0处是因为在0x0:0x80000000之间包含了I/O设备。`_entry`标记的作用是为C代码的运行指定栈，然后就跳转到`start`标记执行C代码。`start`标记的作用是进行一些配置，这些配置只能在机器模式下来做，然后就切换到管理模式从`main`函数开始执行。`start`函数做了两件事，为切换到管理模式做好准备，初始化计时器中断。`main`函数首先是初始化了一些设备和子系统，然后调用`userinit`来创建第一个进程。第一个进程执行了一个小程序`initcode.s`，通过系统调用`exec`执行一个新的程序`/init`。`/init`创建了一个控制台设备文件，并作为文件描述符0，1，2来打开它。然后在无限循环中，启动shell并处理僵尸进程。系统就这样启动了。

3_xv6的页表.md

原文：https://github.com/shzhxh/xv6-riscv-book-CN/blob/master/3_xv6%E7%9A%84%E9%A1%B5%E8%A1%A8.md

操作系统通过页表来向每个进程提供私有的地址空间和内存。页表决定了内存地址的含义，哪些物理内存可以访问。xv6使用页表分隔不同进程的地址空间，并把它们多路复用到单一的物理内存。xv6还使用页表实现了一些技巧：把同一块内存映射到多个地址空间(a trampoline page)，使用未映射的页来保护内核和用户的栈。

分页硬件

RISC-V页表硬件通过把虚拟地址映射到物理地址的方式来把它们连接在一起。

xv6运行Sv39的分页方式，这种分页方式只使用64位虚拟地址里的低39位，高25位并未使用。在这39位有效位里，低12位是offset，剩下27位是index，这意味着每个页表在逻辑上对应着 2^{27} 个PTE(页表入口)。每个PTE占用64位，低54位有效，高10位是保留位。在这54位有效位里，低10位是flags，高44位是PPN(物理页号)。虚拟地址所对应的物理地址是56位的，它的高44位来自于PTE的PPN，它的低12位来自于虚拟地址的offset。

Sv39的页表在物理内存里实际存在的形式是三级树(three-level tree)。树根是个4K的页，包含了512个PTE，每个PTE都对应了第二级的1个页，每个页都包含了第三级的512个PTE。虚拟地址的27位

index里，高9位用来选择树根的PTE，中间9位用来选择二级页表页的PTE，低9位用来选择最终的PTE。

通常情况下，大部分的虚拟地址都是没有映射的。

PTE有10位的flag，可以在kernel/riscv.h看到xv6对于它们的定义。PTE_V的意思是此条PTE是否存在(1:存在，0:不存在)；PTE_R、PTE_W、PTE_X分别代表所对应的页是否可读、写、执行。PTE_U代表用户模式下是否可以访问此页。

内核需要把根页表页的物理地址放到satp寄存器里，硬件才能使用页表。每个CPU都有一个satp寄存器。这意味着每个CPU都可以有自己私有的地址空间，进而运行各自的进程。

内核地址空间

内核有自己的页表。当进程进入内核，xv6切换到内核页表，当从内核返回用户空间，又切换到用户进程的页表。内核的内存是私有的。

文件kernel/memlayout.h描述了内核内存的布局。

QEMU模拟了一个包含I/O设备的计算机。设备的控制寄存器被映射在物理内存0~0x80000000之间。内核通过读写那些内存与设备交互。

内核的大部分虚拟地址都使用的是恒等映射。对于即需要读写虚拟页又需要通过PTE管理物理页的内核来说，这样的直接映射降低了复杂性。只有两处虚拟地址不是直接映射的：

- trampoline页。它被映射在虚拟空间的顶端，它也被用户页表映射在用户空间的同样位置。它被内核虚拟地址空间映射了两次：一次是直接映射，一次是虚拟空间的顶端。

- 内核栈所在的页。每个进程都有自己的内核栈，它被映射在高位，这样在它下面xv6可以保留一个未映射的守护页(guard page)。守护页的PTE是无效的，这就确保了即使内核让内核栈溢出，也仅仅是触发一个错误让内核panic。如果没有守护页，栈溢出将导致执行不正确的指令。与其这样，让内核panic是个更好的选择。

把内核栈和守护页直接映射不可取，因为守护页所对应的物理地址将难以被使用。

对于内核的trampoline页和代码页，使用的权限是PTE_R和PTE_X，因为内核要从那些页里读取和执行指令。对于其它页则使用PTE_R和PTE_W，因为内核要从那些页里读写内存。对于保护页，则要将其设置为无效的。

创建地址空间

管理地址空间和页表的代码主要在kernel/vm.c。核心数据结构是pagetable_t，它是RISC-V根页表页的指针；它要么是内核的页表，要么是某个进程的页表。核心函数是walk，它用来查找一个虚拟地址的PTE，或者用来查找mappages，mappages用来为新的映射安装PTE。以kvm开头的函数用来管理内核页表，以uvm开头的函数则用来管理用户页表，其它函数即可以管理内核页表也可以管理用户页表。copyout用来把数据复制到用户虚拟地址(由系统调用的参数提供)，copyin则是进行相反方向的复制；它们都定义在vm.c里，因为它们需要严格地转换这些地址以找到对应的物理地址。

在启动的早期，main函数调用了kvminit函数来创建内核的页表。此时还没有开启分页，使用的是物理地址。kvminit首先分配了一个物理页用以保存根页表页。然后调用kvmmmap来保存内核所需的转换。这些转

换包括了内核的指令和数据，至到`PHYSTOP`的物理内存，和实际设备的内存范围。

`kvmmap`调用`mappages`来把映射关系保存到页表里。对于每个要映射的虚拟地址，`mappages`通过`walk`来找到对应PTE的地址。然后初始化PTE来保存相关的物理页号，所需的权限(RXV或RWV两种组合)。

`walk`在查找虚拟地址的PTE的时候模仿了分页硬件(paging hardware)。它是三级页表逐层查找。如果查到的PTE无效，说明所需的页不存在；如果参数`alloc`设置为1，`walk`会分配新的页表页，并把它的物理地址放在PTE里。最终返回的是第三级的PTE的地址。

如上代码依赖于物理地址和虚拟地址的直接映射。

`main`调用`kvminithart`来使内核页表生效。它把根页表页的物理地址保存在`satp`寄存器里。然后CPU就要开始地址转换了。由于内核使用了一致性映射，当前虚拟地址的下一条指令将会映射在正确的物理地址上。

每个进程都会分配到一个内核栈。宏`KSTACK`用以生成每个内核栈的虚拟地址，它还为栈保护页(stack-guard pages)预留了空间。`kvmmap`把PTE加到内核页表里，`kvminithart`重载`satp`寄存器这样硬件才能识别新的PTE。

每个RISC-V核都把页表缓存在TLB里。指令`sfence.vma`用以刷新当前核的TLB以使新的页表生效。

物理内存分配

内核必须在运行的时候(run-time)，为页表、用户内存、内核栈、管道缓存分配或释放物理内存。

xv6使用内核之后到`PHYSTOP`之间的物理内存进行运行时(run-time)的分配。它一次性地分配和释放整个4K的页。它通过把空闲页插入一个链表里来保持对这些页的持续追踪。分配就是从链表里移除一个页，释放就是把空闲页再加入到链表里。

物理内存分配器

分配器(allocator)定义在`kernel/kalloc.c`里。分配器的数据结构是由空闲物理页组成的链表。每个空闲页在列表里都是`struct run`。因为空闲页里什么都没有，所以空闲页的`run`数据结构就保存在空闲页自身里。这个空闲列表使用自旋锁进行保护。

`main`函数调用`kinit`来初始化分配器。`kinit`通过保存所有空闲页来初始化链表。xv6其实是可以通过分析配置信息来获取真实内存的大小的，但它没有这么做，而是假定系统只有128M的内存。`kinit`调用`freerange`来把空闲内存加到链表里，`freerange`是通过调用`kfree`把每个空闲页逐一加到链表里来实现此功能的。由于PTE只能指向4K对齐的物理地址，所以`freerange`使用宏`PGROUNDUP`来确保空闲内存是4K对齐的。分配器刚开始是无内存可用的，对`kfree`的调用使得它拥有了可以管理的内存。

分配器有时把地址作为整数以执行一些算法，有时把它们作为指针以读写内存，这就使得分配器的代码里充满了强制类型转换。另外，分配和释放实际上也是改变了内存的类型。

`kfree`首先把内存的每个字节都填充为1，这是为了让之前使用它的代码不能再读取到有效的内容，期望这些代码能尽早崩溃以发现问题所在。然后`kfree`再把页加到空闲列表里，它把`pa`转换为到`struct run`的

指针，把原空闲列表指向 `r->next`，并使空闲列表等价于 `r`。`kalloc` 移除并返回空闲列表的第一个元素。

进程地址空间

当 xv6 在进程间切换的时候，进程的页表也会跟着切换。进程的用户空间从 0 到 `MAXVA`，共计 256G 的内存 (2^{38})。

当进程申请更多内存的时候，xv6 首先用 `kalloc` 分配一个物理页。然后把这个物理页的 PTE 加到进程的页表里。xv6 会设置该 PTE 对应的标志位 (W, X, R, U, V)。

我们可以看到页表有如下好处：1，不同的页表把用户空间映射到不同的物理内存上，这样每个进程都有各自的用户空间。2，从进程的角度看虚拟地址是连续的，但实际上它所对应的虚拟地址却不一定是连续的。3，内核把 trampoline 代码映射到地址空间的顶部，这样单一的物理内存就可以出现在所有的地址空间了。

栈是一个单独的页，它的内容来自于 `exec`。在栈的最顶端，是命令行参数，和它们的指针数组。在这些参数的下面，是 `main` 的入口。

在栈的下面有一个保护页，它被设置为无效的，这样栈溢出的时候就会产生一个 page-fault 的异常。而真实世界的操作系统则可能会在栈溢出的时候给它分配更多的内存。

sbrk

已放在第一章。

exec

已放在第一章。

4_xv6的trap与设备驱动.md

原文：https://github.com/shzhxh/xv6-riscv-book-CN/blob/master/4_xv6%E7%9A%84trap%E4%B8%8E%E8%AE%BE%E5%A4%87%E9%A9%B1%E5%8A%A8.md

有三种情况会使CPU跳出当前指令流转而执行一些指定的代码。第一种情况是系统调用，使用`ecall`指令来让内核做一些事情。第二种情况是异常，是由指令的非法执行引起的。第三种情况是设备中断，由设备发出中断信号引起设备中断。

上述三种情况在这里都被称为trap。当trap发生时，任意正在发生的指令都应当能恢复并继续执行。trap应当是透明的，它的一般流程是：先把控制强制转移到内核，然后内核保存寄存器和一些状态；接下来内核执行相应的处理流程；内核恢复先前保存的状态并从trap返回；原代码恢复继续执行。

xv6内核处理了所有的trap。由于内核本来就是系统调用的提供者，所以它显然处理了系统调用；用户进程需要借助内核来管理设备，所以它也处理了中断；内核会杀死在用户空间产生异常的进程，所以它也处理了异常。

xv6的trap处理例程有四个阶段：一，CPU执行硬件动作；二，一段汇编“向量”用于准备到内核C代码的路径；三，一段C处理代码以决定trap作什么；四，系统调用或设备驱动服务例程。虽然可以用统一的代码路径处理所有的三种trap，但把它分成三种处理路径是有益的：来自内核空间的trap，来自用户空间的trap，计时器中断。

由于内核与设备硬件的交互多是通过中断来进行的，所以设备驱动也和trap放在一起讨论。

RISC-V的trap机构

RISC-V支持许多关于中断的控制寄存器，可以在kernel/riscv.h里看到关于它们的定义。如下是一些重要的寄存器：

- stvec - 保存trap处理例程的地址
- sepc - 当trap发生的时候，保存程序计数器的值。sret指令把sepc的值再保存到pc。内核可以通过修改sepc的值来控制sret返回到哪里。
- scause - 用数字来描述trap的原因
- sscratch - 在trap handler最开始的时候有用
- sstatus - SIE位控制设备中断是否生效，SPP位记录trap来自于哪个模式(用户模式或管理员模式)

如上寄存器只能在管理员模式下操作，不能在用户模式下读写。机器模式下也有同样的一套寄存器，但xv6只把它们用于计时器中断。

除了计时器中断，RISC-V硬件处理所有trap流程都是一样的，如下所示：

1. 如果是设备中断，且sstatus的SIE位为0，则不进行如下流程。
2. SIE位置为0，禁止所有中断。
3. 把pc复制到sepc。
4. 把当前的模式(用户或管理员)保存到sstatus寄存器的SPP位。
5. 设置scause来记录中断的原因。
6. 切换到管理员模式。
7. 把stvec保存到pc。

8. 从新的`pc`开始执行。

CPU必须在一个单独的操作里执行完上述所有的步骤。

要注意的是，CPU不切换到内核的页表，不切换到内核的栈，不保存除`pc`外的其它寄存器。那些事需要由内核来做。在trap的过程中CPU做尽量少的工作，这是为了给软件提供尽可能高的灵活性。

内核空间的trap

xv6内核运行的时候，可能发生两种trap：异常和设备中断。

当内核运行的时候，`stvec`里保存的是标记`kernelvec`的地址(详见`kernel/kernelvec.S`)。由于已经在内核里，所以不需要修改内核页表和内核栈。它只是保存了所有的寄存器，用以从中断代码中恢复。

`kernelvec`保存了被中断内核线程的栈寄存器，因为这个值是属于那个线程的。如果trap引发了不同线程的切换，这就非常重要了。

然后`kernelvec`就跳转到了`kerneltrap`。`kerneltrap`用于处理两种trap：设备中断和异常。`kerneltrap`又通过调用`devintr`来实现其功能。如果是设备中断，则在`devintr`里处理，如果是异常，则`devintr`返回0。因为内核里的异常一般都是严重的错误，所以在`kerneltrap`里对异常的处理就是直接打印相关寄存器的信息然后panic。

如果`kerneltrap`是因为计时器中断被调用的，并且运行的不是调度器线程而是进程的内核线程，`kerneltrap`调用`yield`来给其它线程一个运行的机会。本线程和它的`kerneltrap`也会因为其它线程运行`yield`得以恢复。

当`kerneltrap`完成后它必须能正确的返回。由于`yield`可能会改变`sepc`和`sstatus`的值，所以在刚开始的时候`kerneltrap`要保存它们的值。在最后要恢复它们的值到相应的寄存器才可以从`kerneltrap`返回。返回到`kernelvec`后，从栈弹出值到相应的寄存器并执行`sret`，这将会把`sepc`保存到`pc`从而恢复被中断的内核代码。

当从用户空间进到内核里，xv6会把`kernelvec`保存到`stvec`里(详见`kernel/trap.c`的`usertrap`函数)。这就为内核执行的时候而`stvec`的值是错的留下了窗口时间，在那个窗口关闭设备中断是非常重要的。幸运的是RISC-V开始执行一个trap的时候总是会关闭中断，而且xv6只有在`stvec`寄存器被重新设置后才会使能设备中断。

用户空间的trap

用户空间的trap有三种情况：用户程序的系统调用，非法指令，设备中断。用户空间的trap从`uservec`开始，然后跳转到`usertrap`，然后调用`usertrapret`，然后执行`userret`。

用户代码的trap要比内核的困难多了，因为用户页表没有映射到内核，而且栈指针可能包含无效甚至是有害的内容。

RISC-V硬件并不会在trap发生的时候切换页表，那么就需要把`stvec`指向的trap向量指令映射到用户页表里。此外，trap向量必须切换`satp`以指向内核页表，而且为避免崩溃，向量指令必须在内核页表里映射同样的地址，就像在用户页表里那样。

xv6用一个trampoline页来满足上述约束，trampoline页包含了trap向量代码。xv6在所有的页表里都把trampoline页映射到了同样的虚拟地址。这个虚拟地址就是`TRAMPOLINE`。trampoline的内容来自

于 `trampoline.S`，而且当执行用户代码的时候会把 `uservec` 的地址写入 `stvec`。

`uservec` 开始的时候，所有寄存器里的值都是属于中断代码的。但 `uservec` 需要修改一些寄存器来设置 `satp`，那就需要先把这些寄存器的值保存在内存里。RISC-V 提供 `sscratch` 来帮助做这些事。在 `uservec` 刚开始的 `csrrw` 指令交换了 `a0` 和 `sscratch` 的值。现在 `a0` 里保存的是之内核里 `sscratch` 的值，而 `sscratch` 里保存的是用户代码里 `a0` 的值。

`uservec` 接下来要保存用户寄存器。在进入用户空间之前，内核用 `sscratch` 来指向每个进程的 `trapframe`，`trapframe` 用于保存所有的用户寄存器。由于 `satp` 仍指向用户页表，`uservec` 需要把 `trapframe` 映射到用户地址空间。当创建每个进程的时候，xv6 为进程的 `trapframe` 分配了一个页，并总是把它映射到虚拟地址 `TRAPFRAME`，`TRAPFRAME` 就在 `TRAMPOLINE` 的下面。进程的 `p->tf` 指向 `trapframe`，它虽然是个物理地址，但可以通过内核页表来访问它。

所以，在切换 `a0` 和 `sscratch` 之后，`a0` 里保存的是当前进程 `trapframe` 的指针。现在 `uservec` 保存了所有的用户寄存器，包括在 `sscratch` 里保存的原 `a0` 的值。

`trapframe` 包含了到当前进程的内核栈的指针，当前 CPU 的 `hartid`，`usertrap` 的地址，内核页表的地址。`uservec` 把它们值恢复到相应的寄存器里，然后就调用了 `usertrap`。

`usertrap` 和 `kerneltrap` 的任务是一样的，判断 `trap` 的原因，执行然后返回。首先修改 `stvec` 的值为 `kernelvec` 以处理内核模式的 `trap`。然后保存 `sepc` 的值，因为 `sepc` 的值有可能因为有进程切换进 `usertrap` 而被覆盖。如果 `trap` 是系统调用，用 `syscall` 来处理。如是设备中断，

用 `devinitr` 来处理。否则就是异常，直接杀死错误的进程。系统调用之所以要向保留的用户 `pc` 加4，是为了返回的时候程序计数器指向 `ecall` 的下一条指令。最后，`usertrap` 会检查进程是否已经被杀死，或者如果是计时器中断则应释放CPU。

要想返回用户空间，首先要调用 `usertrapret`。此函数通过设置控制寄存器来准备下一个用户空间的trap。首先，让 `stvec` 指向 `uservec`，然后准备 `uservec` 依赖的 `trapframe` 域，然后给 `sepc` 恢复之前保存的用户程序计数器的值。最后，调用 `trampoline` 页上的 `userret`，由 `userret` 的汇编代码来切换页表。

`usertrapret` 在调用 `userret` 的时候，把进程用户页表的指针传给了 `a0`，把 `TRAPFRAME` 传给了 `a1`。`userret` 把 `satp` 切换为进程的用户页表。由于 `trampoline` 页在所有页表中都映射在同样的虚拟地址，这就是在切换 `satp` 后 `uservec` 能继续执行的原因。接下来 `userret` 为了以后和 `TRAPFRAME` 切换把 `a0` 复制到了 `sscratch`。从此之后，`userret` 可使用的数据只有寄存器里的内容和 `trapframe` 里的内容。接下来 `userret` 把 `trapframe` 里的内容恢复到用户的寄存器，切换 `a0` 和 `sscratch` 的值以恢复用户的 `a0` 寄存器并并为下一次的trap保存 `TRAPFRAME`，最后使用 `sret` 返回用户空间。

计时器中断

xv6使用计时器中断管理时钟和切换进程。`usertrap` 和 `kerneltrap` 调用 `yield` 实现进程切换。时钟硬件的计时器中断附加到每个RISC-V CPU。xv6编程时钟硬件来周期性地中断每个CPU。

RISC-V要求计时器中断发生在机器模式，而不是管理员模式。RISC-V的机器模式没有分页，而且用不同的控制寄存器，所以没有办法在

机器模式下运行普通的xv6代码。所以，xv6处理计时器中断和上述trap机制是完全不同的。

运行于机器模式的`timerinit`(在`kernel/start.c`)，初始化了计时器中断。其中一部分工作是编程CLINT硬件来使某个间隔后生成中断。其它部分是设置一个空白区域，类似于`trapframe`，使得计时器中断的例程保存寄存器，和找到CLINT寄存器的地址。最后，设置`mtvec`为`timerverec`并使能计时器中断。

不管执行的是用户代码还是内核代码，计时器中断可能发生在任何点上；内核也没有办法关闭计时器中断。所以计时器中断在执行的时候必须要保证不能影响到被中断的内核代码。基本的策略是计时器中断让RISC-V产生一个软件中断并立即返回。RISC-V使用通常的trap机制向内核分发软件中断，并允许内核关闭它们。计时器中断产生的软件中断的处理代码在`devintr`。

机器模式的计时器中断向量是`timerverec`。首先，它向空白区域(`scratch area`)保存了一些寄存器的值，这个空白区域是在`start`里被初始化的。然后，告诉CLINT何时生成下一次计时器中断。然后告诉RISC-V生成一个软件中断，恢复寄存器的值并返回。计时器中断里没有C代码。

调用系统中断

本节描述用户代码到`exec`系统调用的路径。

首先，在`user/initcode.S`，把传给`init`的参数放在`a0`和`a1`这两个寄存器里，把`exec`的系统调用号放在`a7`里。系统调用号和系统调用的对应关系可在`kernel/syscall.c`里看到。然后执行`ecall`指令切换到内核并执行`uservec`，`uservec`调用`usertrap`，`usertrap`调用`syscall`。

`syscall`从`trapframe`里获得系统调用号，因为`trapframe`里包含了被保存的`a7`。由于`a7`的值是`SYS_exec`，所以`syscall`将调用`sys_exec`。

`syscall`把系统调用的函数的返回值记录在`p->tf->a0`。当系统调用准备返回用户空间，`userret`将把`p->tf`的值载入寄存器并使用`sret`返回用户空间。于是，当`exec`返回用户空间，系统调用处理例程返回的值就通过`a0`返回了。系统调用通过返回负数表示错误，0或正数表示成功。如果系统调用号无效，`syscall`打印错误并返回-1。

系统调用的参数

关于系统调用的机制，还剩一点需要说明：找到系统调用参数。

RISC-V规定的C调用约定是，通过寄存器来传递参数。在系统调用期间，这些寄存器在`trapframe p->tf`里。函数`arginit`，`argaddr`和`argfd`用于获取系统调用的参数，分别对应整数、指针和文件描述符。它们都调用`argraw`来读取已保存的寄存器。

一些系统调用传递的参数是指针，内核必须用这些指针来读写用户内存。比如，`exec`系统调用传递给内核的参数是字符串指针的数组。这些指针就面临两个挑战。一，用户程序可能是错误的或恶意的，传递给内核的可能是无效指针，或欺骗内核从而访问内核内存。二，内核页表的映射和用户页表的映射是不一样的，内核没法使用普通的指令访问用户空间。

许多内核函数都需要安全地读取用户空间，比如`fetchstr`。系统调用`exec`使用`fetchstr`来获取用户空间的字符串参数。`fetchstr`又调用了`copyinstr`，`copyinstr`查找用户页表里的虚拟地址，把它转化成内核可以使用的地址，再从这个地址里把字符串复制进内核。

`copyinstr`把用户页表`pagetable`的虚拟地址`srcva`里的内容复制到`dst`，最高复制`max`字节。它使用`walkaddr`遍历页表以找到虚拟地址对应的物理地址。由于内核映射了所有的物理地址且是等值映射，所以`copyinstr`可以直接复制字符串。`walkaddr`会检查所提供的虚拟地址是否属于进程的用户地址空间，所以程序无法欺骗内核来读取其它的内存。类似的函数`copyout`，用于把内核的数据复制到用户提供的地址。

设备驱动

驱动是操作系统代码，用于管理特定的设备：它告诉设备硬件去执行操作，配置设备以生成中断，处理因此而产生的中断，与进程交互(进程可能在等待来自设备的I/O)。驱动代码可以很复杂，因为驱动设备和它管理的设备同时执行。另外，驱动程序必须理解设备的硬件接口，这个接口可能是复制的且缺少文档。

需要操作系统注意的设备通常可以配置为生成中断，中断是trap的一种。当设备产生中断的时候内核的trap处理代码必须能识别它，并调用该设备的中断处理例程。在xv6里，这个分发的过程是由`devinitr`来控制的。

许多设备驱动由两部分组成：进程里的代码，中断时运行的代码。进程级的代码是由系统调用来驱动的(像`write`和`read`那样让设备执行I/O)。这部分代码可能是让设备执行一个操作(如读取磁盘里的一个块)，然后等待操作的完成。设备完成操作后发出一个中断。驱动的中断处理例程推算出完成了什么操作(如果存在的话)，在适当的情况下唤醒一个等待的进程，还可能会告诉硬件执行下一个等待的操作。

控制台驱动

以控制台驱动为例来说明驱动程序的结构。控制台驱动通过UART串口来接收人类输入的字符。驱动一次积累一行，处理特殊输入字符(如回退键和Ctrl-u)。用户进程(如shell)，可以使用`read`系统调用从控制台获取输入的行。

驱动管理的UART硬件是16550 chip。在真实的计算机上，16550通过RS232串口连接到终端或其它计算机。

UART通过内存映射寄存器的方法暴露给软件。这意味着，RISC-V通过一些物理地址连接到UART设备，对那些物理地址的读写是在与设备硬件交互，而不是与内存交互。UART的内存映射地址是`UART0`(0x10000000)。那里有少量的UART控制寄存器，每个宽度都是1个字节。那些寄存器被定义在`kernel/uart.c`。例如，`LSR`寄存器是行状态寄存器(line status register)，它有一些位用来表示是否有输入字符需要被软件读取。这些用于读取的字符在`RHR`寄存器(receive holding register)。每读取一个，UART硬件就从内部的等待字符FIFO里删除一个，当FIFO为空时则清空`LSR`的"就绪"位。

xv6的`main`调用`consoleinit`来初始化UART硬件，并配置UART硬件生成输入中断。

xv6的shell通过文件描述符来读取控制台。`read`系统调用通过内核到达`cons_leread`。`cons_leread`等待输入的到达(通过中断)，并缓存进`cons.buf`，然后把输入复制到用户空间，在接收完整行后返回用户进程。如果用户还没有输入完整的行，所有的读进程(reading processes)将在`sleep`调用中等待。

当用户输入一个字符，UART硬件会向RISC-V发送一个中断。依如前所述的中断处理流程，调用到`devintr`。`devintr`通过`scause`寄存器查

到中断来自于外设。然后，通过PLIC来获取是哪个设备中断。如果是UART，则调用`uartintr`。

`uartintr`从UART硬件里读取任何等待输入的字符，并把它们交给`consoleintr`；它不等待字符，因为新的输入会产生新的中断。`consoleintr`的工作是把输入字符积累到`cons.buf`直到接收一个完整的行。`consoleintr`会对回退字符或其它少量字符进行特殊处理。当接收到一个完整的行，`consoleintr`会唤醒等待中的`consoleread`(如果存在)。

一旦被唤醒，`consoleread`将在`cons.buf`里观察到一个完整的行，把它复制进用户空间，并返回到用户空间。

在多核机器上，中断有可能被分配到任意CPU上，这由PLIC管理。当CPU执行的进程在读取控制台的时候，中断也有可能发生。所以，中断处理例程需要设计为无需考虑进程或代码正在中断的情况。

5_xv6的锁.md

原文：https://github.com/shzhxh/xv6-riscv-book-CN/blob/master/5_xv6%E7%9A%84%E9%94%81.md

大多数内核，包括xv6，交错执行多个活动。其中一个交错的资源就是多处理器硬件：多个CPU独立运行的计算机，比如xv6赖以运行的RISC-V。这些CPU共享物理内存，xv6利用这个共享来管理所有CPU读写的数据结构。这个共享提升了如下可能性：一个CPU在读取数据结构而另一个CPU在更新它，多CPU同时更新相同的数据。对那样并行访问的情况如不仔细设计，很可能会产生错误的结果或被破坏的数据结构。即使是单处理器，内核也可能让CPU在多个线程间切换，导致它们交错执行。最后，中断处理例程也可能修改中断代码的数据，这也会导致数据的损坏。**并发**(concurrency)的意思就是指多个指令流交错的情况，由于多处理器并行、线程切换或中断。

内核中充满了并发访问的数据。如，两个CPU可能并发地调用`kalloc`，于是并发地从空闲列表里弹出了元素。内核的设计者倾向于允许大量的并发，因为它可以导致性能和响应能力的提升。然而，内核设计者不得不花大量的努力来证明引入并发后代码的正确性。有很多方法达到正确的代码，一些比较好解释，但另一些不好解释。让并发正确的策略和支持它们的抽象，被称为**并发控制**(concurrency control)。xv6根据情况使用了大量并发控制的技术，更多的并发控制是可能的。本章聚焦于一个广泛使用的技术：锁。

锁提供的是互斥，保证一个时间段内只有一个CPU可以持有锁。如果程序员把所有共享数据都关联了一个锁，且代码在使用那条数据时

候始终持有关联的锁，那么那条数据在一个时间段内只能被一个CPU使用。这种情况下，我们说锁保护了这条数据。

本章其余部分解释了为什么需要锁，怎么实现它，怎么使用它。

竞争条件

为什么需要锁呢？考虑一个链表，它可以被多处理器的任意CPU访问。链表支持push和pop操作，这些操作可能是并发的。xv6的内存分配器很大程度上就是这样工作的，`kalloc`从列表里弹出空闲页，`kfree`把页加到列表里。

在没有处理并发的情况下，如果两个CPU都在执行push操作，有可能两个元素都往同一个位置加，后加的元素会覆盖先加的元素。

上面的情况就是竞争条件(race condition)的一个例子。竞争条件就是并发地访问内存位置，且至少有一个访问是写操作。竞争经常标志着产生了错误，可能是写操作丢失了更新，也可能是读了未完全更新的数据结构。竞争的结果取决于两个CPU的时机和它们在内存系统里的内存操作次序，这使得竞争引起的错误难以复现和调试。比如调试push的时候添加打印语句足以改变执行的时机而使竞争消失。

避免竞争的常用方法就是使用锁。锁保证了互斥，这样一个时间段内只有只有一个CPU可以执行push里的敏感操作，从而使上述设想成为可能。在`acquire`和`release`之间的代码被称为 **临界区**(critical section)。

所谓的锁保护了数据，真的就是指保护了数据所特有的一些**不变性**(invariants)。不变性就是数据的属性，它在整个操作过程中保持不变。一般来说，一个操作的正确行为依赖于操作开始的时候不变性是

真实的。操作可能临时修改不变性但必须在完成前恢复它们。比如，链表例子里，不变性是指列表要指向列表里的第一个元素，并且每个元素的`next`字段要指向它的下一个元素。`push`的实现临时修改了不变性，但随后又消除了这个影响。如上的竞争条件能发生，是因为第二个CPU执行的代码依赖于列表的不变性。正确使用锁以确保一个时间段内只有一个CPU可以操作临界区的数据结构，这样当数组结构的不变性没有被持有的情况下，没有CPU可以执行数据结构的操作。

可以把锁认为是序列化的并发临界区，这样就可以一个时间段只运行一个，并保护了不变性(假定临界区的分隔是正确的)。也可以认为被同一个锁保护的临界区相互之间是原子的，这样只能看到完整修改后的临界区。我们说进程之间相互 **冲突**(conflict)，如果它们同时想要相同的锁，或那个锁发生争用(contention)。

注意在`push`里把`acquire`往前移是正确的。这只是会降低并发性。

代码：锁

xv6里有两种锁：自旋锁和睡眠锁。xv6里的自旋锁是`struct spinlock`。其中最重要的字段是`locked`，为0时表示锁存在，为非0时表示锁被持有。

为避免多CPU同时持有一个锁，需要在临界区内执行原子性的步骤。

RISC-V提供了原子指令`amoswap`，用于原子性地交换内存地址和寄存器的值。当内存地址正在读写时，它使用了特殊硬件来防止CPU使用那个内存地址，从而实现了指令的原子性。

xv6的`acquire`定义在`kernel/spinlock.c`。`acquire`使用了内建函数`__sync_lock_test_and_set`，这个函数最终使用了`amoswap`指令，它返

回的是 `lk->locked` 的原值。 `acquire` 在循环里实现的交换，不断的重试直到获得锁。每个循环都把1交换进 `lk->locked` 并检查之前的值。如果之前的值是0则获得锁，并且交换使得 `lk->locked` 的值为1。如果之前的值是1，表明有其它CPU持有这个锁，把1交换进 `lk->locked` 的操作不生效。

一旦获得锁， `acquire` 为方便调试会记录获得锁的CPU。 `lk->cpu` 字段被锁保护，且只能在持有锁的时候被修改。

`release` 函数是 `acquire` 的反操作，它清空 `lk->cpu` 字段并释放锁。从概念上说，释放锁只需给 `lk->locked` 赋值为0即可。C语言的赋值语句是多条指令完成的，所以它是不具有原子性的。 `release` 使用了内建函数 `__sync_lock_release` 来实现原子性地赋值。这个内建函数最终也使用了 `amoswap` 指令。

代码：使用锁

xv6在很多地方使用了锁。关于它的简单例子，可以看看 `kalloc` 和 `kfree`。如果去掉锁也很难触发错误的行为，这意味着锁的错误和竞争是难以测试的。xv6里应该没有竞争的。

使用锁的难点在于，使用多少锁，锁应该保护哪些数据和不变性。有一些基本原则：一，当一个CPU写变量的时候，其它CPU也可以读或写这个变量，应该使用锁；二，记住锁保护的是不变性，如果一个不变性包含了多个内存位置，所有这些位置都需要被单一的锁保护起来。

上述规则说了什么时候使用锁，但没说什么时候可以不用锁，为了效率不应大量使用锁，因为锁会降低并发性。在并发性不重要的情况下，可以只安排一个线程而不使用锁。简单的内核可以在多处理器上

通过只持有一个锁的方式来实现这点，即在进入内核的时候获得锁，在退出内核的时候释放锁(但在系统调用的时候可能会有问题，如读管道或wait)。许多单处理器操作系统通过这个方法在多处理器上运行，有时被叫做"大内核锁"(big kernel lock)，但这种方法牺牲了并行性，一个时间段内只有一个CPU可以运行。如果内核要进行大型运算，那么使用大量细粒度的锁要高效的多，因为内核是在多个CPU上并行执行的。

xv6的内存分配器是一个粗粒度锁的例子，它只用了一个锁来保护空闲列表。如果不同CPU上的进程在同时分配物理页，大家都要等待在acquire上的自旋(spinning)。自旋降低了性能，因为它是无效的工作。如果对锁的争用浪费了大量的CPU时间片段，可以把分配器设计为有多个空闲列表来提升性能，每个都有自己的锁，这样就可以真正的并行分配了。

作为细粒度锁的例子，xv6为每个文件都分配了锁，这样进程在处理多个文件的时候就不必等待彼此的锁了。文件锁的粒度可以更细，如果想并行地写文件的不同区域的话。最终，锁的粒度需要考虑性能和复杂性两方面的情况。

xv6中的所有锁如下表所示。详细的解释分散在各章之中。

锁	描述
bcache.lock	保护对块缓冲区缓存的分配
cons.lock	序列化访问控制台硬件，避免混合输出
ftable.lock	序列化访问文件列表里的结构体file

锁	描述
icache.lock	保护对inode缓存入口的分配
vdisk_lock	序列化访问磁盘硬件和DMA描述符队列
kmem.lock	序列化内存访问
log.lock	序列化事务日志的操作
pi->lock	序列化管道的操作
pid_lock	序列化next_pid的增长
p->lock	序列化进程状态的改变
tickslock	序列化ticks计数器的操作
ip->lock	序列化每个inode和它的内容的操作
b->lock	序列化每个块缓冲区的操作

死锁和锁序

如果到内核的代码路径必须同时持有多个锁，则所有的代码路径以同样的次序获取这些锁是十分重要的。否则就会有死锁的危险。假定有两个代码路径需要锁A和B，代码路径一按从A到B的次序获取锁，代码路径二按从B到A获取锁。假定线程一执行代码路径一并获取了锁A，线程二执行代码路径二并获取了锁B。接下来，线程一尝试获取锁B，线程二尝试获取锁A。两个获取都会被永远阻塞，因为它们分别持有了对方需要的锁，并且在获取所需的锁之前都不会释放自己的锁。

全局性的锁获取次序意味着，锁是函数规范的一部分：调用者在调用函数的时候，必须使锁按规定的次序被获取。

xv6包含很多长度为2的锁序链(lock-order chains)，包含每个进程的锁(struct proc里的锁)，这是由于sleep的工作方式。例如，consoleintr是处理输入字符的中断例程。当新行到达时，等待控制台输入的所有进程都应被唤醒。为实现此目的，consoleintr在调用wakeup的时候持有了锁cons.lock，获取等待进程的锁是为了唤醒它。因此，全局的避免死锁的锁序包含了一个规则，即必须在任何进程锁之前先获取cons.lock。xv6里最长的锁链(lock chains)在文件系统的代码里。例如，创建文件需要同时目录持有锁，新文件的inode持有锁，磁盘块缓冲区持有锁，磁盘驱动的vdisk_lock，和调用进程的p->lock。为避免死锁，文件系统代码总是按前述的次序获取锁。

实现一个全局性的避免死锁的序列可能极为困难。有时锁序和逻辑程序结构矛盾，比如代码模块M1调用M2，但锁序要求M2的锁在M1的锁之前获取。有的时候锁的身份无法预先知道，有可能是因为先持有一个锁才能发现这个锁的身份。这个情况出现在文件系统中，当它在路径名中查找连续组件的时候，在exit和wait的代码里当在进程表里搜索子进程的时候。最后，死锁的风险经常是对细粒度锁制定锁方案的限制，因为更多的锁意味着更多的死锁机会。避免死锁是内核实现的一个重要因素。

锁和中断处理例程

一些xv6自旋锁保护的数据被线程和中断处理例程两者使用。例如，在clockintr增加ticks的时候内核正在sys_sleep里读取ticks。锁tickslock序列化了这两个访问。

自旋锁和进程的交互提升了潜在的危险性。考虑`sys_sleep`持有`tickslock`，且它的CPU发生了计时器中断。`clockintr`尝试获取`tickslock`，看见它被持有了，一直等待这个锁的释放。这种情况下，`tickslock`永远不会被释放，只有`sys_sleep`能释放它，但`clockintr`不返回`sys_sleep`就不能继续执行。所以，CPU会死锁，任何需要锁的代码都会被冻结。

为避免这种情况，如果中断处理例程使用了自旋锁，当中断发生的时候CPU一定不能持有那个锁。xv6更保守一点，CPU获取锁之前会关闭那个CPU上的所有中断。其它CPU仍然能发生中断，所以一个中断的`acquire`仍然可以等待一个线程来释放自旋锁，只是不在同一个CPU而已。

当CPU不持有任何自旋锁的时候，xv6会重新开启中断，它必须做点簿记(book-keeping)来复制嵌套的临界区。`acquire`调用`push_off`，`release`调用`pop_off`来记录当前CPU上锁的嵌套层级。当记数为0，`pop_off`将恢复临界区最外层的中断状态。`intr_off`和`intr_on`分别执行RISC-V指令来关闭和开启中断。

`acquire`在设置`lk->locked`之前直接调用`push_off`是非常重要的。如果颠倒了两者的次序，当中断使能的情况下持有锁会有一个明显的窗口，这时如果发生计时器中断将锁死系统。同样地，`release`也要在释放锁后调用`pop_off`。

指令与访存排序

认为程序执行的次序就如源代码所显示的那样是很自然的。但是很多编译器和CPU为了实现更高的性能不按次序执行代码。对于需要多个周期才能完成的指令，CPU可能会提前发射这个指令，让它与其它指

令重叠，以避免空等。例如，CPU在一个指令序列里可能会发现A和B不相互依赖。CPU可能会先执行指令B，也许是因为A的输入之前它的输入已经就绪，也许是为了重叠执行A和B。编译器也可能进行类似的重新排序。

编译器和CPU按其它次序执行要遵守的规则是，不能改变正常次序下执行的结果。但是，这个规则会改变并发代码的结果，并且很容易引发多处理器上的错误行为。CPU的次序规则被称为**内存模型**(memory model)。

如果编译器或CPU把临界区的代码放到临界区外执行，这就会引发灾难。

为了避免那样的问题，xv6在`acquire`和`release`里使用了`__sync_synchronize()`。`__sync_synchronize()`是一个内存屏障(memory barrier)：它告诉编译器不要跨屏障重新排序load和store。xv6的`acquire`和`release`的屏障在几乎所有重要的情况下强制了次序，因为xv6使用锁访问共享数据。

睡眠锁

有时xv6需要长时间持有锁。例如文件系统在读写一个文件在硬盘上的内容的时候要保持它的锁，这样的磁盘操作可能需要几十个毫秒。那么长时间持有自旋锁将导致浪费，因为其它想获取这个锁的进程会长时间地浪费CPU。自旋锁的另一个缺点是当进程持有锁的时候不可以让出CPU，我们会希望持有锁的进程在等待磁盘的时候其它进程可以使用这个CPU。持有自旋锁的时候让出CPU是非法的，因为其它线程想获取这个自旋锁的时候会触发死锁；因为`acquire`不让出CPU，第二个线程的自旋可能会阻止第一个线程运行和释放锁。持有锁的时候

让出CPU也违背了持有自旋锁的时候必须关闭中断这一规定。所以我们就需要有一种锁，当等待获取的时候让出CPU，且持有锁的时候也允许yield和中断。

xv6提供了那样的锁，即**睡眠锁**。`acquiresleep`在等待的时候让出CPU，它使用的技术详见“调度”那一章。从较高的层面来看，睡眠锁有一个通过自旋锁保护的`locked`字段，`acquiresleep`调用`sleep`自动让出CPU并释放自旋锁。结果是`acquiresleep`等待的时候其它线程得以执行。

因为睡眠锁让中断打开了，它们不可以在中断处理例程中使用。因为`acquiresleep`可能让出CPU，睡眠锁不可以在自旋锁的临界区内使用(但自旋锁可以在睡眠锁的临界区内使用)。

自旋锁适合于短的临界区，因为等待它们要浪费CPU时间；睡眠锁适合于长的操作。

6_xv6的调度.md

原文：https://github.com/shzhxh/xv6-riscv-book-CN/blob/master/6_xv6%E7%9A%84%E8%B0%83%E5%BA%A6.md

任何操作系统的进程都可能比CPU多，所以进程需要分时共享(time-share)CPU。理想状态下，共享对用户进程应当是透明的。常用的方式的把进程多路复用到硬件CPU上，使每个进程产生拥有自己的虚拟CPU的假象。本章描述xv6如何实现的多路复用。

多路复用

xv6的多路复用是让进程在所有CPU之间切换，它分两种情况。一，当进程等待设备或管道I/O的完成，或等待子进程退出，或在sleep系统调用中等待，使用sleep和wakeup机制切换。二，定时强制切换进程。多路复用使得每个进程看上去都拥有自己的CPU，就像内存分配器和硬件页表的使用使得每个进程看上去都拥有自己的内存一样。

实现多路复用会有一些挑战。一，怎么来让进程切换呢？虽然上下文切换的想法很简单，它的实现却是xv6里最不透明的代码。二，如何让强制切换对用户进程透明？xv6使用计时器中断驱动上下文切换，这是标准技术。三，多个CPU并发地在进程间切换，需要锁来避免竞争。四，进程退出时它的内存和其它资源必须被释放，但它不能自己完成全部的过程，比如它不能在运行的时候释放自己的内核栈。五，多核系统的每个核都必须记住它运行的是哪个进程，这样系统调用才能影响正确进程的内核状态。最后，sleep和wakeup允许放弃CPU并睡眠以等待一个事件，并允许其它的进程唤醒先前的进程。要小心避免

竞争，否则会导致唤醒通知的丢失。xv6尝试尽可能简单地解决这些问题，但最后的代码依然很复杂。

代码：上下文切换

从一个进程切换到另一个进程的步骤是：用户进程通过系统调用或中断进入内核态，通过上下文切换进到当前CPU的调度线程，再通过上下文切换进入到新进程的内核线程，返回到用户态的进程。xv6调度器在每个CPU上都有一个专用的线程(保存的寄存器和栈)，因为它运行在任何进程的内核栈上都是不安全的。本节中将研究内核线程和调度器线程之间的切换机制。

从一个线程切换到另一个包含了保存旧线程的寄存器，和恢复新线程的寄存器。切换sp寄存器意味着要切换栈，切换pc寄存器意味着切换要执行的代码。

函数`swtch`实现了线程切换时保存和恢复两个过程。`swtch`并不知道线程，它只是保存和恢复寄存器的集合，这个寄存器的集合叫做**上下文**。当进程要让出CPU的时候，进程的内核线程调用`swtch`来保存它自己的上下文并返回到调度器的上下文。每个上下文都包含在`struct context`里，而它自己又被包含在进程的`struct proc`里或CPU的`struct cpu`里。`swtch`有两个参数：`struct context *old`指当前的寄存器组，`struct context *new`指要载入的寄存器组。

我们来看看一个进程是怎么通过`swtch`进到调度器的。中断导致`usertrap`执行，`usertrap`调用`yield`。`yield`再调用`sched`，`sched`调用`swtch`将当前上下文保存在`p->context`，并切换到之前保存在`cpu->scheduler`的调度器上下文。

`swtch`只保存被调用者要保存的寄存器，调用者的寄存器由调用的C代码保存在栈上(如果需要的话)。`swtch`知道`struct context`里每个寄存器字段的位置。它不保存`pc`寄存器。相反，`swtch`保存`ra`寄存器，这个寄存器保存着调用`swtch`的返回值。现在`swtch`从新上下文里恢复寄存器，新上下文里保存的是上一个`swtch`的寄存器。当`swtch`返回，它返回到`ra`所指向的指令处，也就是新线程之前调用`swtch`的指令。除此之外，它返回到新线程的栈上。

在我们的例子里，`sched`调用`swtch`以切换到`cpu->scheduler`。上下文通过`scheduler`对`swtch`的调用而得以保存。当我们追踪的`swtch`返回时，它返回到`scheduler`而不是`sched`，并且它的`sp`寄存器将指向当前CPU的调度器的栈。

代码：调度

上一节讨论了`swtch`的底层细节，本节研究一个进程是怎么通过调度器切换到另一个进程的。调度器就是每个CPU上运行的`scheduler`函数。这个函数的任务就是选择下一个要运行的进程。一个进程要让出CPU首先要获取它自己的进程锁`p->lock`，释放它持有的其它锁，更新它自己的状态`p->state`，然后调用`sched`。当我们在做`sleep`和`exit`的时候，`yield`就遵循了这个约定。`sched`再次检查了这些条件，并检查这些条件隐含的结果：如果持有锁，中断应当是关闭的。最后，`sched`调用`swtch`保存`p->context`的上下文并切换到`cpu->scheduler`里调度器的上下文。`swtch`返回到调度器的栈，就像`scheduler`的`swtch`已经返回了。调度器继续`for`循环，找到一个进程来运行，切换到它，重复这个过程。

我们在调用`swtch`的过程中`xv6`持有了`p->lock`：`swtch`的调用者必须持有锁，并且对锁的控制传递给了要切换过去的代码。这个约定在锁上是

不常见的，通常哪个线程获取锁，哪个线程就负责释放锁，这有利于判断正确性。对于上下文切换来说，则要打破这个约定，因为 `p->lock` 保护的是进程的 `state` 和 `context` 字段的不变量，那些字段在执行 `swtch` 的时候是不正确的。如果在 `swtch` 期间不持有 `p->lock` 可能会有这样的问题：对于在 `yield` 之后已经把自己的状态设置为 `RUNNABLE` 的进程，可能会有其它CPU决定运行它，但此时 `swtch` 可能还没有使它停止使用它自己的内核栈。这导致两个CPU运行在同样的栈上，这是错误的。

内核线程总是在 `sched` 里让出CPU，并且总是在调度器里切换到同样的位置，调度器几乎总是切换到之前调用 `sched` 的内核线程上。线程切换实际上就是 `scheduler` 和 `sched` 的 `swtch` 交替执行的过程。这种在两个线程间程式化切换的过程叫做**协程**(coroutines)。 `sched` 和 `scheduler` 就互为协程。

在一种情况下，调度器调用 `swtch` 但不在 `sched` 中结束。当一个新的进程第一次调度的时候，它在 `forkret` 开始。 `forkret` 的存在就是为了释放 `p->lock`，否则新进程就应从 `usertrapret` 开始。

`scheduler` 运行的是一个简单的循环：找到一个进程来运行，直到这个进程让出CPU，然后一直重复这样的过程。调度器在进程表里找到一个可运行的进程，那个进程是 `p->state==RUNNABLE`。一旦它找到这样的进程，它就设置当前CPU的当前进程 `c->proc`，将进程标记为 `RUNNING`，然后调用 `swtch`。

可以把调度代码的结构认为是强制每个进程的一组不变量，一旦这些不变量是不正确的就持有 `p->lock`。一个不变量是，如果一个进程是 `RUNNING` 的，中断计时器的 `yield` 必须可以安全地切换出去；这意味着CPU的寄存器必须保留着进程寄存器的值(如 `swtch` 没有把它们移动到一个 `context`)，且 `c->proc` 必须指向这个进程。另一个不变量是，如

如果一个进程是`RUNNABLE`的，它必须是安全的以便一个空闲CPU的`scheduler`可以运行它；这意味着`p->context`必须保留进程的寄存器(如，它们不是在真实的寄存器里)，没有CPU运行在进程的内核栈上，没有CPU的`c->proc`指向此进程。显然当持有`p->lock`这些特性都不正确。

xv6之所以在一个线程里请求`p->lock`而在另一个线程里释放它，就是为了保持上面所提的不变量，比如在`yield`请求而在`scheduler`释放。一旦`yield`开始一个运行进程的状态使它`RUNNABLE`，必须持有锁直到不变量被恢复：最早的正确释放点在`scheduler`清空`c->proc`之后，那时`scheduler`运行在它自己的栈上。同样地，一旦`scheduler`开始把一个可运行的进程转换为`RUNNING`，直到内核线程完全运行的时候才可以释放锁(对于`yield`来说就是在`swtch`之后)。

`p->lock`也保护了其它的东西：`exit`和`wait`的相互作用，避免唤醒(wakeup)的缺失，避免一个进程退出时其它进程读写它的状态的竞争(如，`exit`系统调用查找`p->pid`并设置`p->killed`)。出于清晰和性能的考虑，也许应该把`p->lock`的不同功能分开。

代码：mycpu和myproc

xv6经常需要一个指针来指向当前进程的`proc`结构体。单处理器上可以用一个全局变量来指向当前`proc`。然而在多核机器上却不行，因为每个核都在执行不同的进程。解决这个问题的方法是利用每个核自己的寄存器；可以利用其中的一个寄存器来查询那个核的信息。

xv6为每个CPU维护了一个`struct cpu`，它记录了在那个CPU上正在运行的进程(如果存在的话)，为CPU的调度器线程保存的寄存器，用于管理中断关闭的自旋锁嵌套层数。函数`mycpu`返回当前CPU

的`struct cpu`的指针。RISC-V给所有CPU进行了编号，即`hartid`。xv6确保每个CPU的`hartid`保存在那个CPU的`tp`寄存器里。这使得`mycpu`可以使用`tp`来索引`cpu`结构体的数组以找到正确的那个。

确保一个CPU的`tp`总是保存着这个CPU的`hartid`有点复杂。在CPU启动的初期`start`设置了`tp`寄存器，当时仍在机器模式。`usertrapret`把`tp`保存在trampoline页，因为用户进程可能修改`tp`。最后，当从用户空间进到内核的时候`uservec`恢复已保存的`tp`。编译器保证绝不使用`tp`寄存器。如果RISC-V允许xv6直接读取当前`hartid`会更方便，但只能是在机器模式，在管理员模式是不允许的。

`cpuid`和`mycpu`的返回值是脆弱的：如果发生计时器中断使得线程让出CPU，返回值将不再正确。为避免这个问题，xv6需要调用者关闭中断，只有在使用完返回的`struct cpu`后才可以开启中断。

函数`myproc`返回`struct proc`，它指向的是在当前CPU正在运行的进程。`myproc`关闭中断，调用`mycpu`，从`struct cpu`获取当前进程的指针`c->proc`，然后使能中断。即使开启中断`myproc`的返回值也是安全的：如果计时器中断把调用的进程移动了其它CPU，它的`struct proc`指针仍然是不变的。

睡眠与唤醒

调度和锁有助于一个进程相对于其它进程隐藏自己的存在性，但到目前为止还没有抽象方式可以帮助进程之间的交互。许多机制被发明出来解决这个问题。xv6使用的机制叫睡眠和唤醒，这允许一个进程睡眠以等待一个事件，一旦这个事件发生其它进程会把这个进程唤醒。睡眠和唤醒被称为**序列协调**(sequence coordination)或**条件同步**(conditional synchronization)。

为了说明我们的意思，考虑一个简单的同步机制**信号量**(semaphore)，它被用来协调生产者和消费者。信号量包含了一个记数，并提供了两个操作。V操作递增记数(用于生产者)，P操作等待记数为0，然后递减记数并返回(用于消费者)。如果生产者线程和消费者线程都只有一个，且它们在不同CPU上运行，且编译器没有进行太积极的优化，如下实现是正确的：

```
struct semaphore {
    struct spinlock lock;
    int count;
}

void V(struct semaphore *s){
    acquire(&s->lock);
    s->count += 1;
    release(&s->lock);
}

void P(struct semaphore *s){
    while(s->count == 0)
        ;
    acquire(&s->lock);
    s->count -= 1;
    release(&s->lock);
}
```

如上实现代价高昂。如果生产者很少活动，消费者就要花大量的时间在while循环里自旋。相对于忙等，消费者的CPU应该去做一些更有意义的工作。要想避免忙等，就需要消费者让出CPU且只在V操作后恢复。

让我们来考虑一对调用sleep和wakeup。sleep(chan)睡在任意值chan上，被等待频道(wait channel)调用。sleep让调用的进程睡眠，释放CPU来干其它的事情。wakeup唤醒所有睡眠在chan上的进程(如果存在的话)，引发它们的sleep调用返回。如果没有进程等待在chan上，wakeup就什么也不做。可以把wakeup加到V操作，且把sleep加到P操作，来改变信号量的实现。

P操作不再自旋而是放弃了CPU，这很好。但是，设计sleep和wakeup这样的接口并不容易，因为它会带来所谓的**唤醒丢失**(lost wake-up)问题。假定P操作发现s->count == 0。当P操作还没有执行sleep的时候，V运行在了其它CPU上：它改变了s->count为非0的值并调用了wakeup，它发现没有睡眠的进程所以什么都没有做。现在P操作继续执行sleep并进程睡眠。这就引发了一个问题：P进入睡眠等待一个已经发生了的V操作。除非我们很幸运生产者又调用了V操作，否则消费者将永远等待(被死锁了)即使记数是非0的。

问题的根源在于，当s->count == 0时P才能睡眠的不变量被破坏了，因为V运行在错误的时候。可以在P里使用自旋锁让计数检查和sleep调用是原子的。这样的P确实可以避免唤醒丢失，但它依然会死锁：P在睡眠的时候持有锁，导致V等待这个锁而被永远阻塞。

我们通过修改sleep接口修复之前的方案：调用者必须把锁传递给sleep，这样在调用进程被标记为睡眠且等待在睡眠频道之后，sleep就可以把锁释放。锁将强制并发的V等待直到P完成自己的睡眠，这样wakeup将发现睡眠的消费者并把它唤醒。一旦消费者被再次唤醒，sleep在返回前会请求锁。

P持有s->lock避免了P在检查c->count和调用sleep期间，V唤醒它的尝试。我们需要sleep来原子性地释放锁并让消费者进程睡眠。

代码：睡眠与唤醒

我们来看看sleep和wakeup的实现，它们都在kernel/proc.c。基本思路是：让sleep把当前进程标记为SLEEPING并调用sched让出CPU；wakeup查找在等待频道上睡眠的进程并把它标记为RUNNABLE。sleep和wakeup的调用者可以使用任何方便的数字作为频道。xv6经常使用与等待相关的内核数据结构的地址。

sleep请求p->lock。现在准备睡眠的进程持有p->lock和lk。调用者持有lk是必需的(在本例中是P)：它确保没有其它进程(在本例中是运行的V)可以调用wakeup(chan)。现在sleep持有p->lock，释放lk就是安全的：其它进程可能开始调用wakeup(chan)，但wakeup将会等待获取p->lock，这样就一直等到sleep完成让进程睡眠，使得wakeup不会丢失sleep。

有一个小问题：如果lk是像p->lock一样的锁，当sleep尝试获取p->lock的时候会把自己死锁。但如果调用sleep的进程已经持有了p->lock，它并不需要做其它的事来避免丢失并发的wakeup了。当wait带着p->lock调用sleep时会出现这种情况。

现在sleep仅持有p->lock，它可以通过记录睡眠频道、改变进程状态和调用sched来让进程睡眠。

稍后，某个进程将调用wakeup(chan)。wakeup在进程表里循环。它获取它检查的每个进程的p->lock，即是因为它可以管理进程的状态，也是因为p->lock不会让sleep和wakeup彼此错过。当它发现一个SLEEPING状态的进程与chan匹配的时候，它把进程状态改为RUNNABLE。下次调度器运行的时候，它会发现这个进程已经准备好运行了。

xv6代码持有"条件锁"的时候总是会调用`wakeup`；在信号量的例子里那个锁是`s->lock`。严格来说如果`wakeup`总是跟在`acquire`之后就足够了(即，应该在调用`release`后调用`wakeup`)。为什么`sleep`和`wakeup`的锁的规则要确保睡眠的进程不会丢失它需要的唤醒呢？睡眠的进程，从它检查条件之前到它标记为睡眠之后，要么持有条件锁，要么持有自己的`p->lock`，要么持有两者。如果一个并发的线程使得条件为真，那个线程必需持有这个条件锁，或在睡眠的线程请求条件锁之前，或在睡眠的线程在`sleep`里释放它之后。如果是在之前，睡眠线程必须看到新的条件值，并且不管怎样决定睡眠，所以它不担心丢失唤醒。如果是在之后，唤醒者可以请求条件锁的最早时机是在`sleep`请求`p->lock`之后，所以`wakeup`对`p->lock`的请求必须等待直到`sleep`让睡眠者彻底睡眠。然后`wakeup`将看到睡眠的进程并把它唤醒(除非有什么东西先把它唤醒)。

可能会有多个进程睡在同一个频道上的情况；例如，多个进程读取一个管道。一旦调用`wakeup`将会把它们全部唤醒。其中一个进程将首先执行并请求`sleep`被调用时所请求的锁，并读取所有等待在管道中的数据。其它进程会发现，尽管被唤醒了，但无数据可读。从它们的角度来看，这个唤醒是“假的”，它们必须再次睡眠。因此，`sleep`总是在检查条件的循环中被调用。

如果在两次使用“睡眠/唤醒”的时候不小心选择了同样的频道，不会造成任何伤害：它们会看到假的唤醒，但上面所述的循环可以容忍这样的问题。“睡眠/唤醒”的魅力在于它们都是轻量级的(不需要创建特殊的数据结构来充当睡眠频道)，并提供了一个间接层(调用者不需要知道它在和哪个进程交互)。

代码：管道

使用`sleep`和`wakeup`来同步生产者和消费者的一个复杂的例子，是xv6里对管道的实现。数据从管道的一端写入，复制进内核里的缓冲区，然后从管道的另一端读出。我们先来看看`pipewrite`和`piperead`的实现。

每个管道都代表了一个`struct pipe`，它包含了一个`lock`和一个`data`缓冲区。字段`nread`和`nwrite`记录读出或写入缓冲区的字节数。缓冲区的封装：`buf[PIPE_SIZE-1]`的下一个字节是`buf[0]`。计数不封装。这个约定让一个满的缓冲区(`nwrite == nread + PIPE_SIZE`)和一个空的缓冲区(`nwrite == nread`)区分开来，但是它也使得缓冲区的索引必须是`buf[nread % PIPE_SIZE]`而不是简单的`buf[nread]`(对于`nwrite`来说也是一样)。

假定两个不同的CPU同时调用了`piperead`和`pipewrite`。`pipewrite`从请求管道的锁开始，这样可以保护记数、数据和相关的不变量。`piperead`也试着请求锁，但请求不到。它在`acquire`上自旋等待这个锁。当`piperead`等待的时候，`pipewrite`循环遍历要写入的字节(`addr[0...n-1]`)，依次将每个字节添加到管道中。在循环的过程中，缓冲区可能被填满。在这种情况下，`pipewrite`调用`wakeup`来提醒睡眠的读者有数据等待在缓冲区，并睡在`&pi->nwrite`上来等待一些读者从缓冲区中取走一些数据。`sleep`释放`pi->lock`，这是让`pipewrite`的进程进入睡眠的一部分。

现在`p->lock`是可用的，`piperead`可以请求它并进到它的临界区：它发现`pi->nread != pi->nwrite`(`pipewrite`因为`pi->nwrite == pi->nread+PIPE_SIZE`而进入睡眠)，所以它进到`for`循环里，从管道中复制出数据，按复制出的字节数递增`nread`。现在有许多字节可以写入了，所以`piperead`在它返回前调用`wakeup`唤醒睡眠的写

者。`wakeup`找到睡在`&pi->nwrite`上的进程，这个进程就是那个因为满了而中断的`pipewrite`。它把那个进程标记为`RUNNABLE`。

管道代码为读者和写者使用了不同的睡眠频道(`pi->nread`和`pi->nwrite`)；当有大量的读者和写者等在相同的管道上时(当然这种情况不太可能发生)，这可能会使系统更高效。在检查睡眠状态的循环里，管道代码睡眠了；如果有多个读者或写者，除了第一个进程外，所有被唤醒的进程都发现条件是假的并继续睡眠。

代码：wait, exit和kill

`sleep`和`wakeup`可以用在多种类型的等待中。一个有趣的例子是，一个子进程的`exit`和它的父进程的`wait`交互。在子进程死亡时，父进程可能已经在`wait`里睡眠了，或者在做其它的事；在后一种情况下，一个随后的对`wait`的调用必须发现子进程死亡了，可能在子进程调用`exit`很久之后。xv6记录子进程死亡的方式，是调用`exit`让子进程进入`ZOMBIE`状态，直到父进程的`wait`发现它才把状态改为`UNUSED`，复制子进程的退出状态，并把子进程的进程号返回给父进程。如果父进程先于子进程退出了，父进程把子进程给到`init`进程，`init`不停地调用`wait`；因此，每个子进程都有一个父进程来清理。代码实现的最大挑战是可能的竞争和死锁。

`wait`使用调用进程的`p->lock`作为条件锁来避免唤醒丢失，它在一开始的时候就请求了那个锁。然后它描述进程表。如果它发现了处于`ZOMBIE`状态的子进程，它释放那个子进程的资源 and 它的`proc`结构体，把子进程的退出状态复制到提供给`wait`的地址(如果它不是0的话)，并返回子进程的进程号。如果`wait`发现了有子进程但这些子进程都没有退出，它会调用`sleep`等待其中一个退出，然后重新开始描述。在这里，`sleep`释放的条件锁是`p->lock`，即上面提到的特殊情况。注

意`wait`经常会持有两个锁，为避免死锁，它使用的锁序是先父进程后子进程。

`wait`使用`np->parent`的时候不持有`np->lock`，这违反了通常的规则，即共享变量必须用锁保护起来。因为`np`有可能是当前进程的祖先，在这种情况下请求`np->lock`可能会引起死锁(违背了先父进程后子进程的锁序)。在这种情况下，不使用锁检查`np->parent`看起来是安全的；一个进程的`parent`字段只能被它的父进程改变，所以如果`np->parent==p`为真，这个值不会改变除非当前进程改变它。

`exit`记录退出状态，释放一些资源，把子进程给到`init`进程，唤醒处于`wait`中的父进程，将调用者标记为僵尸进程，并永久性地让出CPU。最后的顺序有点复杂。退出的进程在把自己的状态设为`ZOMBIE`的时候必须持有它的父进程的锁，并且把父进程唤醒，因为父进程的锁是条件锁，这个条件锁用来防止在`wait`里的唤醒丢失。子进程还必须持有它自己的`p->lock`，否则父进程可能发现它处于`ZOMBIE`状态从而在它仍然运行的时候释放它。锁请求的次序对于避免死锁来说是非常重要的：因为`wait`先请求父进程的锁再请求子进程的锁，所以`exit`必须使用同样的次序。

`exit`调用一个特殊的唤醒程序`wakeup1`，它只唤醒睡在`wait`上的父进程。看上去子进程在把自己的状态设为`ZOMBIE`之前就唤醒父进程是不正确的，但实际上这很安全：虽然`wakeup1`可能引发父进程的运行，`wait`里的循环不会检测子进程直到子进程的`p->lock`被`scheduler`释放，所以`wait`不会看到退出的进程直到`exit`把它的状态设为`ZOMBIE`。

`exit`允许一个进程终止自己，`kill`则允许一个进程终止其它的进程。`kill`如果直接毁掉目标进程将会非常复杂：因为目标进程可能正在其它CPU上运行，可能正在对内核的数据结构进行一系列的敏感更

新。`kill`只做了少量的工作就解决了这些挑战：它只是设置目标进程的`p->killed`，如果目标进程在睡眠就唤醒它。最终目标进程会进入或离开内核，那时如果设置了`p->killed`，在`usertrap`会调用`exit`。如果目标进程运行在用户空间，它也会因为系统调用或计时器中断而进入内核。

如果目标进程在`sleep`，`kill`调用`wakeup`会让目标进程从`sleep`返回。这有潜在的风险，因为正在等待的条件可能不是真的。然而xv6调用`sleep`问题包装在一个`while`循环中，这个循环会在`sleep`返回后重新测试条件。一些对`sleep`的调用也会在循环里测试`p->killed`，如果设置了则终止当前的活动。只有在那种终止是正确的情况下才可以这么做。例如，如果设置了`killed`标志管道的读写代码会返回；最终会返回到`trap`，然后又开始检查标志并退出。

一些xv6的`sleep`循环不检查`p->killed`，因为代码正在进行多步的系统调用，那是原子性的。`virtio`驱动就是一个例子：它不检查`p->killed`，因为为了让文件系统保持正确的状态，操作操作可能是所有必需的写操作里的其中一个。一个标记为`killed`的进程在等待磁盘I/O的时候不会退出，直到它完成当前的系统调用，`usertrap`看到`killed`标记。

7_xv6的文件系统.md

原文：https://github.com/shzhxh/xv6-riscv-book-CN/blob/master/7_xv6%E7%9A%84%E6%96%87%E4%BB%B6%E7%B3%BB%E7%BB%9F.md

文件系统的目标是组织和保存数据。文件系统应支持在用户和程序之间共享数据，它也应该支持持久性(persistence)使得在重启之后数据也是可用的。

xv6的文件系统支持像Unix那样的文件、目录和路径名，并把数据保存在一个virtio硬盘上以支持持久性。文件系统解决了许多挑战：

- 文件系统需要磁盘上的数据结构来代表文件和目录的树，记录块的标识(块上记录了所有文件的内容)，记录硬盘的哪些区域是空闲的。
- 文件系统必须支持故障恢复(crash recovery)。即，如果发生了故障(如，电源错误)，文件系统必须在重启之后仍然能正确工作。风险在于故障可能中断一系列的更新并使硬盘上的数据结构不一致(如，一个块可能即被一个文件使用也被标记为空闲)。
- 可能会有不同进程同时操作文件系统，因此文件系统的代码必须协调以维护不变量。
- 访问磁盘要比访问内存慢几个数量级，因此文件系统必须维护常用块在内存里的缓冲区。

概述

xv6的文件分为七层。如下图所示：

名称	描述
文件描述符层	用文件系统抽象了许多Unix资源(管道、设备、文件等)，应用程序员好过多了
路径名层	提供分层的路径名且用递归查找来解析它们
目录层	目录是特殊的inode，它的内容是一些目录的入口，每个入口包含了文件名和i-number
inode层	提供用 <code>inode</code> 代表的独立文件，且有唯一的i-number和一些保存了文件数据的块
日志层	让上层在一个事务中封装对多个块的更新，且确保发生故障时这些块是原子性地更新(如，全更新或全不更新)
缓冲区缓存层	缓存硬盘块并同步对它们的访问，确保每次只有一个内核进程可以修改块上的数据
硬盘层	读写virtio硬盘上的块

文件系统必须有一个在磁盘上存储inode和内容块的方案。xv6的作法是把磁盘分为多个区。文件系统不使用0号块(它是启动区)。块1是超级块(superblock)，包含了文件系统的元数据(文件系统里块的数量，数据块的数量，inode的数量，日志区里块的数量)。从2号块开始是日志区。日志区之后是inode区，每个块有多个inode。之后是位图区，

用于追踪哪个块被使用了。剩下的是数据区，数据区里的块要么在位图区被标记为空闲，要么保存文件或目录的内容。超级块由一个叫mkfs的单独的程序填充，这个程序构建了一个初始的文件系统。

硬盘层

硬盘层在virtio_disk.c，实际上就是硬盘驱动。

缓冲区缓存层用到的接口是virtio_disc_rw

- 作用：对磁盘进行读或写
- *b：缓冲区的指针
- write：1把缓冲区的内容写入到磁盘，0把磁盘的内容读取到缓冲区

缓冲区缓存层

缓冲区缓存有两个工作(代码在bio.c)：

1. 同步访问磁盘块以确保内存里每个块只有一份复制，且每次只有一个内核线程可以使用那份复制。
2. 缓存常用块使得不必每次都从硬盘上读取它们。

缓冲区缓存的主要接口是bread和bwrite：bread获取一个buf来包含一个块的复制，这样就可以在内存里进行读写；bwrite把缓冲区中的内容写入到相应的块里。内核线程必须通过调用brelse来释放一个缓冲区。缓冲区缓存使用每个缓冲区的睡眠锁来确保一次只有一个线程可以使用每个缓冲区(进而每个硬盘块)；bread返回一个上锁的缓冲区，然后brelse释放那个锁。

我们再回到缓冲区缓存。缓冲区缓存有一个确定数量的缓冲区来保存磁盘块，这意味着如果文件系统请求了不在缓存中的磁盘块，缓冲区缓存必须回收一个已经保存了其它块的缓冲区。缓冲区缓存回收的是最近最少使用的缓冲区。

代码：缓冲区缓存

缓冲区缓存是缓冲区的双向链表。`main`调用`binit`来初始化这个列表，列表中的项来自于静态数组`buf`里的`NBUF`个缓冲区。所有对缓冲区缓存的其它访问都是通过`bcache.head`引用链表来实现的，而不是`buf`数组。

缓冲区缓存的物理表现形式是数组，逻辑表现形式是双向链表。每个缓冲区都有三个部分组成，`data`字段标示了它的内容，指针的字段用于组成链表，数值字段用于标示它的属性（如与磁盘和块的对应关系，如当前缓冲区是否记录了对应磁盘的内容等）。

在日志层里用到的本层接口有：

- `bread`
- `bwrite`
- `brelse`

与缓冲区相关的状态字段有两个。字段`valid`的意思是那个缓冲区包含了一个块的复制。字段`disk`的意思是缓冲区的内容已经被提交到了磁盘，这可能会改变缓冲区(如，把磁盘中的数据写到`data`)。

`bread`调用`bget`来为给定的扇区分配缓冲区。如果缓冲区需要从磁盘读取数据，在返回前`bread`会调用`virtio_disk_rw`从磁盘把数据读出来。

`bget`通过设备和扇区号在缓冲区列表里扫描对应的缓冲区。如果有那样的缓冲区，`bget`请求那个缓冲区的睡眠锁，然后把带锁的缓冲区返

回给调用者。如果没有对应的缓冲区，`bget`必须生成一个，可能还需要新使用一个已保存了数据的缓冲区。它会第二次扫描缓冲区列表，查找没有被使用的缓冲区(`b->refcnt == 0`)；任何一个那样的缓冲区都是可用的。`bget`编辑缓冲区的元数据来记录新设备、扇区号并请求它的睡眠锁。注意赋值语句`b->valid = 0`，它确保了`bread`将从磁盘读取块数据，而不是错误地使用缓冲区里之前的数据。

每个磁盘扇区最多只有一个缓冲区是十分重要的，这样可以确保读者可以看到写操作，并且文件系统也需要在缓冲区上使用锁来进行同步。通过从第一个循环里检查这个块是否被缓存，到第二循环里声明这个块已经被缓存(设置`dev`，`blockno`和`refcnt`)，`bget`连续地持有`bcache.lock`来确保这个不变量。这使得对块存在性的检查和(如果不存在)为保存那个块而进行的缓冲区分配是原子的。

`bget`在`bcache.lock`临界区之处请求缓冲区的睡眠锁是安全的，因为`b->refcnt`非0使得那个缓冲区不会被重新使用。睡眠锁保护了对块缓冲区内容的读写，而`bcache.lock`保护了哪个块被缓存的信息。

如果所有的缓冲区都忙，表明有太多的进程并发地执行文件系统调用，`bget`就会panic。一个更优雅的反应可能是睡眠来等待一个缓冲区空闲，尽管那样可能会引发死锁。

一旦`bread`读取了磁盘(如果需要的话)并把缓冲区返回给它的调用者，调用者就独占了缓冲区的使用，进行读写数据了。如果调用者修改了缓冲区，它在释放缓冲区之前必须调用`bwrite`来把改变的数据写入到磁盘。`bwrite`调用`virtio_disk_rw`来与磁盘硬件对话。

如果调用者完成了对缓冲区的操作，它必须调用`brelse`来释放它。`(brelse`是`b-release`的缩写，它名字有点怪但值得学习一下：源自于

Unix，在BSD、Linux和Solaris也都是这么用的)。

`brelse`释放睡眠锁，并把缓冲区移到链表的前面。对缓冲区的移动引发了列表按照最近使用(释放)进行排序：列表里的第一个缓冲区是最近被使用的，最后一个则是最近最少被使用的。`bget`里的两个循环利用了这点：在最坏的情况下，扫描一个已经存在的缓冲区必须处理整个列表，当引用处于良好的位置的时候，先检查最近使用的缓冲区将减少扫描时间。通过反向扫描(跟随`prev`指针)，对要重新使用的缓冲区的扫描查找到了最近使用的缓冲区。

日志层

在文件系统的设计里最有趣的问题之一是故障恢复。许多文件系统的操作包含了多次写磁盘的操作，在一串写操作之后的崩溃使得磁盘上的文件系统处于不一致的状态。比如，发生在文件裁切时的崩溃(设置文件长度为0且释放它的内容块)。依赖于写磁盘的次序，崩溃可能使引用到一个内容块的inode被标记为空闲，也可能使一个内容块被分配但未被引用。

后者相对来说是良性的，但引用到一个空闲块的inode在重启后可能会引发严重的问题。重启后，内核可能把那个块分配给了其它文件，现在两个不同的文件都指向了同一个块。如果xv6支持多用户，这就会产生安全问题，因为旧文件的属主可以读写新文件的块，而这个新文件的拥有者是不同的用户。

xv6通过简单的日志记录解决了文件系统操作期间的崩溃问题。一个xv6系统调用不直接写硬盘上文件系统的数据结构。相反，它把一个描述放在磁盘上，这个描述是它在一个`log`里所期望的所有磁盘写操作。一旦系统调用日志记录了所有的写操作，它往磁盘上写入一个特

殊的`commit`记录用来表示那个日志包含了一个完整的操作。那时系统调用才会把复制写入到磁盘文件系统里的数据结构。当那些写操作都完成后，系统调用删除磁盘上的日志。

如果系统崩溃并重启，在运行任何进程之前，文件系统代码按如下描述从崩溃中恢复。如果日志被标记为包含一个完整的操作，则恢复代码把写操作复制到磁盘文件系统。如果日志不是标记为包含完整的操作，恢复代码忽略这个日志。恢复代码最后删除日志完成所有的操作。

为什么xv6的日志解决了在文件系统操作期间的崩溃问题？如果崩溃发生在操作提交之前，则磁盘上的日志不会标记为已完成，恢复代码会忽略它，磁盘状态就像操作从来没有开始过一样。如果崩溃发生在磁盘操作提交之后，则恢复代码将会重新执行所有的写操作，如果已经开始往磁盘数据结构中执行写操作，则可能会重新执行这些操作。不管是哪种情况，相对于崩溃来说日志都使得操作原子化了：恢复之后，要么所有的写操作都出现在磁盘上，要么它们都不出现在磁盘上。

日志设计

日志驻留在固定的位置，这是在超级块里指定的。它是由一个头块(header block)和随后跟着的一些要更新的块的复制(updated block copies)组成的，要更新的块的复制也叫日志块(logged blocks)。头块由扇区编号的数组和日志块的计数组成，其中每个扇区对应着一个日志块。头块中的计数如果是0则表示日志中没有事务，如果非0则表示日志中包含一个完整的已提交事务。xv6在事务提交的时候(不是之前)写头块，在把日志块复制到文件系统之后将计数置0。因此事务进行

到一半的崩溃将导致头块里计数为0；而提交之后的崩溃将导致计数非0。

上面一段描述了磁盘上日志区的结构，是理解日志层的关键。

每个系统调用代码都意味着，相对于崩溃来说，写序列的开始和结束是原子的。为了允许不同进程对文件系统操作的并发执行，日志记录系统可以把多个多个系统调用的写操作累积到一个事务中。即一个提交可能包含多个完整系统调用的写操作。为了避免在事务之间分割系统调用，日志系统只有在没有文件系统的系统调用发生时才提交。

把多个事务一块提交的想法被称为组提交(group commit)。组提交减少了磁盘操作的数量，因为它把一次提交的固定代价分摊到了多个操作上。组提交还有助于磁盘系统进行并发的写操作，这使得磁盘在一次旋转期间就可能把它们全部写入。xv6的virtio驱动不支持这样的批处理(batching)，但xv6文件系统的设计允许这样做。

xv6在磁盘上占用固定的空间来保存日志。在一个事务中系统调用要写入的块的数量必须适合那个空间。这有两个后果。

- 不允许单个的系统调用写入比日志空间更多的块。对于大多数系统调用来说不存在这样的问题，但是write和unlink可能会写许多块。一个大文件的写操作可能会许多数据块、许多位图块和一个inode块；取消到一个大文件的连接可能会写许多位图块和一个inode块。xv6的write系统调用把大的写操作分解为多个小的写操作以适合日志，unlink不会产生问题因为xv6的文件系统实际上只使用一个位图块。
- 日志系统不可以允许系统调用启动，除非确定系统调用的写操作符合日志的剩余空间。

代码：日志

日志层的代码在log.c里。数据结构log可以看做是由两部分组成的，其中lh记录了磁盘上日志区里头块的内容，其它整数则代表了一次事务的属性（如当前事务包含了多少个系统调用，如当前事务是否在提交等）。

本层向上层提供的接口是：

- begin_op
- log_wirte
- end_op

在系统调用里log的典型用法如下所示：

```
begin_op();  
...  
bp = bread(...);  
bp->data[...] = ...;  
log_write(bp);  
...  
end_op();
```

`begin_op`一直在等待，直到日志系统当前没在提交，且直到有足够的日志空间来保存调用的写操作。`log.outstanding`记录保留了日志空间的系统调用的数量；总的保留空间等于`log.outstanding`乘以`MAXOPBLOCKS`。递增的`log.outstanding`即保留了空间也防止了在这个系统调用期间发生提交。代码里谨慎地假设每个系统调用都可能写入超过`MAXOPBLOCKS`个块。

`log_write`是**`bwrite`**的代理。它在内存中记录块的扇区号，在硬盘上的日志中为它保留一个位置，并把缓冲区固定在块缓存里以防止块缓存

驱逐它。块必须待在缓存里，直到提交：在那之前，被缓存的拷贝是修改的唯一记录；在提交之前，不得写入磁盘；且同一事务中的其它读操作必须可以看到这个修改。当一个块在单个事务中被多次写入时，`log_write`会发现它，并在日志里为那个块分配相同的位置。这个优化被称为**合并**(absorption)。这很常见，比如，在一个事务里包含了多个文件的inode的磁盘块被多次写入。通过把多个磁盘写操作合并到一个，文件系统可以节省日志空间并获得更好的性能，因为只需将一个磁盘块的拷贝写到磁盘里。

`end_op`首先对未完成的系统调用的计数进行递减操作。如果记数为0，则调用`commit()`提交当前事务。这个过程分为四步。

1. `write_log()`把在事务中修改过的每个块从缓冲区缓存复制到硬盘上日志分区对应的位置里。
2. `write_head()`把头块写到硬盘上。这是提交点，写操作之后的崩溃将导致从日志中恢复事务的写操作。
3. `install_trans`从日志中读取每个块并把它写入到文件系统的对应位置。
4. 把计数0写到日志的头块。必须在下一个事务写日志头块之前做这件事，这样当崩溃发生的时候，就不会产生头块属于一个事务而后续的日志块属于另一个事务的问题。

在第一个用户进程运行之前，`fsinit`调用`initlog`，`initlog`调用`recover_from_log`。它读取日志头，如果日志头表示如果日志包含了一个已提交的事务，则模仿`end_op`的动作。

在`filewrite`里有一个使用日志的例子。这个事务像下面这样：

```
begin_op();
ilock(f->ip);
r = writei(f->ip, ...);
iunlock(f->ip);
end_op();
```

这个代码被包装在一个循环中，把大的写操作拆分成多个独立的事务，以避免日志溢出。对`writei`的调用写了许多块，这也作为事务的一部分：文件的inode，一个或多个位图块，和一些数据块。

代码：块分配器

文件和目录的内容保存在磁盘块上，必须从空闲池中分配这些磁盘块。xv6的块分配器管理着一个磁盘上的空闲位图，每个位对应着一个块。值为0代表着对应的块是空闲的，值为1代表着对应的块在被使用。程序`mkfs`设置各种块对应的位，这些块有启动扇区、超级块、日志块、inode块，和位置块。

块分配器提供了两个函数：`balloc`分配一个新的磁盘块，`bfree`释放一个块。

- `balloc`里的循环考虑到了所有的块，从0到`sb.size`，文件系统里所有的块。它查找位图里的位为0的块，一旦找到就更新位图并返回这个块。为提高效率，这个循环被分为两个部分。外层循环读取位图区里的每个块。内层循环检查单个位图块里的所有`BPB`个比特。如果两个进程同时分配同一个块，可能会发生竞争，但缓冲区缓存一次只允许一个进程使用一个位图块，这就避免了这种竞争。
- `bfree`找到正确的位图块，并清空正确的位。此外，`bread`和`brelease`里暗含的排它性，避免了显式地使用锁。

就像在本章其余部分描述的大部分代码一样，`balloc`和`bfree`必须在一个事务的内部调用。

inode层

术语`inode`有两个相关的含义。

- 磁盘上的数据结构，包括文件大小和磁盘块编号的列表。
- 内存里的数据结构，包含一份磁盘上inode的拷贝以及内核需要的额外信息。

磁盘上的inode被填充在一个被称为inode块的连续磁盘区域上。每个inode都是一样大小，所以给定一个编号n，很容易就找到磁盘上的第n个inode。事实上，编号n(被称为inode编号或i-number)，是在具体实现中inode的标识方式。

磁盘上的inode通过`struct dinode`来定义。`type`字段区分了文件、目录、和特殊文件(设备)。如类型值为0则表示磁盘inode是空闲的。字段`nlink`记录了引用当前inode的目录条目的数量，用以识别这个磁盘inode和它的数据块何时应该被释放。`size`字段记录了文件内容的字节数。`addrs`数组记录了保存了文件内容的磁盘块的块号。

内核把活动inode的集合保存在内存中，即`struct inode`。只有在一个C指针指向一个inode的时候内核才会把那个inode保存到内存里。`ref`字段记录了C指针引用内存里inode的次数，当那个计数降为0的时候内核就会从内存中丢弃这个inode。`iget`和`iput`函数请求和释放到一个inode的指针，这会修改这个引用计数。到inode的指针可能来自于文件描述符、当前的工作目录，和事件内核代码(如`exec`)。

xv6的inode代码里有四种锁或类似于锁的机制。

- `icache.lock`保护的不变量是inode最多在缓存中出现一次，以及`inode.ref`计数的是到被缓存的inode的指针数量。
- `inode.lock`是inode的睡眠锁，它用来确保对inode的字段、inode的文件或目录内容块的独占访问。
- `inode.ref`如大于0，则使系统保留缓存里的这个inode，并且不会让其它的inode重新使用这个缓存条目。
- `inode.nlink`，用于计数引用了一个文件的目录条目的数量，如果一个inode的连接数量(link count)大于0，xv6是不会释放这个inode的。

`iget()`返回的`struct inode`指针保证是有效的，直到调用相应的`iput()`；这个inode不会被删除，指针所引用的内存也不会被也不会其它的inode重新使用。`iget()`提供对inode的非独占访问，所以可以有多个指针指向同一个inode。文件系统代码里的许多部分都依赖于`iget()`的这个行为，即可以保持对inode的长期引用(当打开文件和当前目录)，又可以避免竞争，同时还可以在操作多个inode(如路径名查找)的时候避免死锁。

`iget()`返回的`struct inode`可能没有任何有用的内容。为了确保它确实保存了磁盘inode上的拷贝，代码必须调用`ilock`。这会锁定inode(所以其它进程就无法`ilock`它了)，并从磁盘读取尚未读取的inode。`iunlock`则释放inode上的锁。在某些情况下，把inode指针的获取与锁定分享有助于避免死锁，例如在目录查找期间。多个进程可以持有`iget()`返回的inode的C指针，但一次只有一个进程可以锁定这个inode。

inode的缓存里缓存的，是内核代码或数据结构持有C指针的inode。它的主要工作是支持多进程的并行访问，缓存只是次要的功能。如果

一个inode被频繁使用，且它不是由inode缓存保存，它可能会被缓冲区缓存保存在内存里。inode缓存是直写的(write-through)，这意味着修改了被缓存的inode的代码必须用 `iupdate` 立即写入磁盘。

代码：inode

在fs.c里。

xv6调用 `ialloc` 来分配一个新的inode(如创建文件)。 `ialloc` 类似于 `balloc`：它遍历磁盘上的inode结构，每次一个块，查找标记为空闲的块。当它找到空闲块，它就把新的 `type` 写入磁盘的方式来声明这个块，然后通过调用 `iget` 返回一个inode缓存的条目。 `ialloc` 的正确操作依赖于这样的事实，一次只有一个进程可以持有对 `bp` 的引用： `ialloc` 就可以确保其它进程不会同时发现这个inode的存在并尝试声明它。

`iget` 遍历inode缓存来查找一个活动的入口(`ip->ref > 0`)，这个入口要满足设备和inode编号两个要求。如果找到了这样的入口，它返回对那个inode的新的引用。当 `iget` 扫描的时候，它会记录第一个空位的位置，这个空位用于分配一个缓存条目(如果需要的话)。

在读写inode的元数据或内容之前，必须使用 `ilock` 来锁定inode。 `ilock` 使用一个睡眠锁来实现此目的。一旦 `ilock` 可以独占地访问这个inode，它会根据需要从磁盘(更有可能是缓冲区缓存)读取这个inode。函数 `iunlock` 释放这个睡眠锁，这可能导致任何正在睡眠的进程被唤醒。

`iput` 通过检查引用计数(reference count)释放一个inode的C指针。如果这是最后一次引入，在inode缓存里这个inode的位置就是空闲的并且可以被其它inode重新使用了。

如果`iput`发现一个inode没有C指针引用了，也没有到它的连接(不存在于任何目录中)，则这个inode和它的数据块必须被释放。`iput`调用`itrunc`来把文件截为0字节，释放数据块；把inode的类型设为0(未分配)；并把inode写入磁盘。

在`iput`释放inode的时候，它里面的锁定规则(the locking protocol)值得仔细研究。

- 其中一个风险是一个并发线程为了使用这个inode可能会等待在`ilock`里(如，读取一个文件或列出一个目录)，并且不会发现这个inode已经不再被分配了。这不会发生，如果一个缓存的inode的连接数为0而引用数为1，一个系统调用是没有办法获取到它的指针的。这1个引用被调用`iput`的线程所拥有。`iput`确实是会在它的`icache.lock`临界区之外检查引用计数为1，但那时连接数已经是0，所以没有线程会尝试获取新的引用。
- 另一个主要的风险是对`ialloc`并发的调用可能会关闭inode，而这个inode已经被`iput`释放掉了。在`iupdate`写磁盘之后，从而inode的类型为0时，这个风险才有可能发生。这个竞争是良性的；在读写inode之前，分配线程会礼貌地等待以请求inode的睡眠锁，此时`iput`已经完成了它。

`iput()`可以写入磁盘。这意味着任何使用了文件系统的系统调用都可以写磁盘，因为系统调用可能是最后一个到文件的引用。即使是像`read()`那样看上去只读的系统调用，也有可能会最后调用`iput()`。这意味着，任何使用了文件系统的系统调用，即使它们是只读的，也必须封装在事务中。

`iput()`和崩溃之间的交互具有挑战性。当文件的连接数降为0的时候，`iput()`不会立即的截断文件，因为仍然可能有进程在内存中持有

到这个inode的引用：一个进程可能仍然在读写这个文件，因为它成功打开了这个文件。但是，如果在最后一个进程关闭这个文件的文件描述符之前发生了崩溃，这个文件就会标记为分配到了磁盘上，却没有目录条目指向它。

文件系统有两种方法来处理这种情况。

1. 简单点的方案是，在重启之后恢复的时候，寻找那些已经标记为分配但没有目录条目指向它们的文件。如果找到那样的文件就把它释放。
2. 这个方案不必扫描整个文件系统。如果一个文件的连接数降为0而引用数不为0，则把这个文件的inode编号记录到磁盘上(比如，超级块上)。当引用记数降为0的时候删除这个文件，再更新这个磁盘列表以从列表中删除那个inode。恢复的时候，文件系统释放列表中的所有文件。

xv6以上两个方案都没有使用，这意味着inode标记为在磁盘上已分配，即使它们不再被使用。这意味着随着时间的推移，xv6面临着耗尽磁盘空间的风险。

代码：inode的内容

硬盘上的inode结构`struct dinode`，其中`size`字段用以表明文件大小，`addrs`数组用以保存块号。那些块号所对应的块里保存着这个inode的数据。`addrs`里的前`NDIRECT`个块被称为**直接块**(direct blocks)；第`NDIRECT+1`个块记录了`NINDIRECT`个块的数据，它被称为**间接块**(indirect block)。由于一个块的大小`BSIZE`是1024字节，且`NDIRECT`是12，所以一个文件可以直接载入的内容为12k字节。由于`NINDIRECT`是256，所以读取间接块后可以载入的内容是256k字节。这有利于在磁

盘上的表达，但对于用户程序来说比较复杂。函数**bmap**管理这个表达。**bmap**用于返回第**bn**个数据块的硬盘块号，参数**ip**用于表示inode的指针。如果**ip**里没有那个块，**bmap**会给它分配一个。

函数**bmap**先从简单的情况开始：前**NDIRECT**个块已经在inode它自己里列出了。接下来的**NINDIRECT**个块在间接块**ip->addrs[NDIRECT]**里列出。**bmap**读取间接块，然后再从间接块里读取对应的块号。如果块号超过了**NDIRECT+NINDIRECT**，**bmap**会panic；**writei**会进行检查以避免这种情况的发生。

bmap按照需要来分配块。如果直接块或间接块的某个条目为0，则意味着那个条目没有分配块。当**bmap**发现条目的值为0，它就把新块的编号填充进去，按需分配。

itrunc释放一个文件的块，把inode的大小重置为0。**itrunc**先释放直接块，再释放间接块里所列出的那些块，最后再释放间接块自身。

对于**readi**和**writei**，**bmap**让它们获取inode的数据变的简单。**readi**首先确保位移和大小没有超出文件的限制。读取的位移超出文件的限制会返回一个错误，读取的大小超出文件的限制则会返回少一些的字节。主循环处理文件的所有块，把缓冲区的数据复制到**dst**。**writei**和**readi**相同，但有三个地方不一样：

1. 写入的大小超出文件的限制会返回错误。
2. 循环里不是读出，而是把数据复制进缓冲区。
3. 如果写操作扩展了文件，**writei**必须更新它的大小。

在xv6-public的代码里，**readi**和**writei**都是从检查**ip->type==T_DEV**开始。但在xv6-riscv的代码里，没有进行这样的检查。

函数`stat`把inode的元数据复制进`stat`结构体，`stat`结构体最终会通过`stat`系统调用暴露给用户程序。

代码：目录层

```
in fs.c
```

目录的内部实现非常类似于文件。它的inode的类型是`T_DIR`，它的数据是一些目录条目。每个条目都是一个`struct dirent`，包含了一个名字和一个inode号。名字最多可以有`DIRSIZ`(14)个字符；如果低于14个字符，则以`NUL`(0)结尾。inode号为0的目录条目是空闲的目录条目。

函数`dirlookup`通过名字来查找目录里的条目。如果找到一个，它返回对应inode的指针(未锁定)，并把`*poff`设置为目录里对应条目的位移，以防调用者期望能编辑它。如果`dirlookup`找到了正确名称的条目，它更新`*poff`并返回未上锁的inode(通过`iget`获取)。`iget`返回不上锁的inode就是因为`dirlookup`。调用者锁定了`dp`，所以如果查找的是`.`(当前目录的别名)，返回之前锁定inode会导致尝试重新锁定`dp`从而导致死锁。还有更复杂的死锁涉及多进程和`..`（上级目录的别名）。调用者可以解锁`dp`然后锁定`ip`，确保它一次只持有一个锁。

函数`dirlink`向目录`dp`里写入一个新的目录条目，参数`*name`是新目录条目的名称，参数`inum`是inode号。如果名称已经存在，`dirlink`返回一个错误。主循环读取目录条目查找一个未分配的条目。当它找到一个，它就先停止循环，此时`off`被设置为已知条目的位移。否则，`off`将被设置为`dp->size`。不管是哪种情况，`dirlink`都把新的条目写入位移`off`，这样新条目就填加到了目录里。

代码：路径名

路径名查找包含了一系列对`dirlookup`的调用，一个`dirlookup`对应了路径名的一个部分。`namei`计算`path`并返回对应的`inode`。函数`nameiparent`是一个变量：它在最后一个元素之前停止，返回上级目录的`inode`并把最后的元素复制进`name`。它们都是调用`namex`来完成实际的工作的。

`namex`首先决定路径计算从哪里开始。如果路径是从斜杠开始的，计算从根目录开始；否则，就从当前目录开始。然后它使用`skipelem`来依次得到路径里的每个元素。每次循环都必须在当前`inode(ip)`查找`name`。循环首先锁定`ip`并检查它是否是一个目录。如果不是，查找失败。(锁定`ip`是必要的，不是因为`ip->type`可以改变，它改变不了，而是因为在`ilock`运行之前，不能保证已经从磁盘加载了`ip->type`。)如果这个调用是`nameiparent`并且这是最后一个路径元素，按照`nameiparent`的定义，循环终止；最后的路径元素已经复制进了`name`，所以`namex`只需返回未锁定的`ip`即可。最后，使用`dirlookup`查找路径元素，并通过设置`ip = next`为下一次循环做好准备。当循环遍历完所有的路径元素退出后，`namex`返回`ip`。

例程`namex`可能需要很长的时间才能完成：它可能包含了多个磁盘操作来读取`inode`和目录块。xv6经过仔细的设计，使得如果一个内核线程调用`namex`的时候被阻塞在磁盘I/O上，其它内核线程可以并发的查找不同的路径名。`namex`给不同路径下的目录分别上锁，使得在不同目录下的查找可以并发地进行。

这样的并发带来一些挑战。例如，一个内核线程正在查找一个路径名，而其它内核线程可能正在改变这个目录树(通过取消链接一个目

录)。一个潜在的风险是，一个查找可能在搜索已经被其它内核线程删除的目录，并且这个目录的块已经被其它目录或文件重新使用了。

xv6避免了那样的竞争。例如，在`namex`里执行`dirlookup`的时候，查找线程持有了目录的锁，并且`dirlookup`返回一个来自于`iget`的inode。`iget`递增这个inode的引用计数。只有在从`dirlookup`接收到这个inode之后，`namex`才会释放这个目录的锁。现在其它线程可能会从目录里取消链接这个inode，但是xv6不会删除这个inode，因为这个inode的引用计数仍然大于0。

另一个风险是死锁。例如，当查询`.`目录的时候，`next`指向与`ip`相同的inode。在释放对`ip`的锁之前锁定`next`应该会引发死锁。为了避免这种死锁，`namex`在获取`next`锁之前会先解锁当前目录。在这里我们再次看到了`iget`和`ilock`分离的重要性。

文件描述符层

in file.c

Unix接口一个酷的地方在于一切皆是文件，包括设备(如控制台)、管道、和真正的文件。文件描述符层实现了这个特性。

每个进程有自己的打开文件(或者说是文件描述符)的表。每个打开文件都是一个`struct file`，它是对inode或管道的封装，再加上一个I/O位移。每次对`open`的调用都创建一个新的打开文件(一个新的`struct file`)：如果不同的进程各自打开同一个文件，不同的实例将会有不同的I/O位移。另一方面，一个单独的打开文件(相同的`struct file`)可能在一个进程的文件表里多次出现，也可以出现在多个进程的文件表里。如果一个进程使用`open`打开文件，然后使用`dup`创建别名，或者使用`fork`与子进程共享这个文件，就会发生这种情况。

引用计数追踪一个特定的打开文件的引用数量。`readable`和`writable`字段追踪的是一个文件以什么方式打开(读, 写, 或读写)。

系统里的所有打开文件保存在一个全局文件表里, 即`ftable`。可以操作这个文件表的函数有: 分配文件(`filealloc`), 创建一个重复的引用(`filedup`), 释放一个引用(`fileclose`), 读写数据(`fileread`和`filewrite`)。

- `filealloc`扫描文件表查找未引用的文件(`f->ref == 0`)并返回一个新的引用。
- `filedup`递增引用计数。
- `fileclose`递减引用计数。当引用计数降为0, 释放底层对应的管道或inode。
- `filestat`实现了对文件的`stat`操作。它只允许操作inode, 然后调用`stati`
- `fileread`实现了对文件的`read`操作。首先检查是否允许`readable`模式, 然后把调用传递给管道或inode的实现。如果文件是一个inode, 它使用I/O位移作为读操作的位移, 然后增加这个位移。管道没有位移的概念。
- `filewrite`实现了对文件的`write`操作。首先检查是否允许`writable`模式, 然后把调用传递给管道或inode的实现。如果文件是一个inode, 它使用I/O位移作为写操作的位移, 然后增加这个位移。管道没有位移的概念。要记得inode函数需要调用者管理锁定。inode的锁定有一个方便的附加效果, 即读写位移是自动更新的, 因此同时对一个文件的多个写操作不会覆盖彼此的数据, 但这些写操作可能会交错在一起。

代码：系统调用

in sysfile.c

底层实现了大部分功能，系统调用这一级的实现是微不足道的。只有少数的系统调用值得研究一下。

函数`sys_link`和`sys_unlink`编辑目录，创建或删除对inode的引用。它们是好的例子来展现使用事务的强大。`sys_link`从获取它的两个字符串参数`old`和`new`开始。如果`old`存在且不是目录，递增它的`ip->nlink`计数。然后调用`nameiparent`来找到`new`的父目录和最终的路径元素，并创建一个新的目录条目来指向`old`的inode。新的上级目录必须存在且和inode在同一设备上：inode号在单独的磁盘上有唯一的意义。如果发生了这样的错误，`sys_link`必须回退且递减`ip->nlink`。

事务简化了这个实现，因为它需要更新多个磁盘块，但我们不必担心更新的次序。要么全部更新，要么全不更新。如果没有事务，在创建一个连接之前更新`ip->nlink`，可能会让文件系统临时进入一个不安全的状态，此时的崩溃可能导致严重的破坏。如果有了事务我们就不必担心这点了。

`sys_link`为一个存在的inode创建了一个新的名称。函数`create`为一个新inode创建一个新的名称。它是三个系统调用的通用操作：`open`使用`O_CREATE`标志创建一个新的普通文件，`mkdir`创建一个新的目录，`mkdev`创建一个新的设备文件。像`sys_link`一样，`create`首先调用`nameiparent`来获取上级目录的inode。然后调用`dirlookup`检查名称是否存在。如果名称存在，`create`的行为依赖于它是被哪个系统调用使用：对于`mkdir`和`mkdev`来说`open`有不同的语义。如果`create`是代表`open(type == T_FILE)`使用的，并且名称存在且是一个常规文件，那么`open`就算是成功了，`create`也是。否则，就是一个错误。如果名称不存在，`create`使用`ialloc`创建一个分配个新的inode。如果新的inode

是一个目录，`create`使用`.`和`..`条目初始化它。最后，数据被正确初始化了，`create`把它链接进上级目录。`create`像`sys_link`一样，同时持有了两个inode锁：`ip`和`dp`。没有死锁的可能，因为inode`ip`是最新分配的：系统中没有其它进程会持有`ip`的锁，所以也就不会尝试锁定`dp`。使用`create`，可以轻松实现`sys_open`、`sys_mkdir`和`sys_mknod`。`sys_open`是最复杂的，因为创建一个新文件只是它的工作的一小部分。如果给`open`传递`O_CREATE`，它会调用`create`。否则，它调用`namei`。`create`返回一个上锁的inode，但`namei`不会，所以`sys_open`必须自己锁定这个inode。这可以方便地检查目录是为读而打开的，不是写。假定inode是这样或那样获得的，`sys_open`分配一个文件和一个文件描述符，然后填满这个文件。注意没有其它进程可以访问部分初始化的文件，因为它只存在于当前进程的表中。

在调度那一章里，在没有文件系统前，就研究了管道的实现。函数`sys_pipe`通过提供创建管道对的方法连接到文件系统的实现。它的参数是一个数组指针，数组里有两个整数，它将在里面记录两个新的文件描述符。然后它分配管道并保存文件描述符。

8_xv6之重新审视并发.md

原文：https://github.com/shzhxh/xv6-riscv-book-CN/blob/master/8_xv6%E4%B9%8B%E9%87%8D%E6%96%B0%E5%AE%A1%E8%A7%86%E5%B9%B6%E5%8F%91.md

9_磁盘的结构.md

原文：https://github.com/shzhxh/xv6-riscv-book-CN/blob/master/9_%E7%A3%81%E7%9B%98%E7%9A%84%E7%BB%93%E6%9E%84.md

磁盘的结构

[boot block | sb block | log | inode blocks | free bit map | data blocks]

- 超级块里有一个魔数用来验证文件系统
- 超级块里保存了日志区，inode区，位图区的起始位置
- 超级块里保存了日志区，inode区，数据区和整个镜像的大小

函数的功能

```
/*
 * 作用： 依据块的使用情况写位图区
 * int  : 已经使用的块的数量
 * 原理：
 */
void balloc(int);

/*
 * 作用：向指定扇区写入数据。
 * sec  : sect No.
 * *buf  : 要写入扇区的数据。
 * 原理：首先用lseek系统调用定位到扇区所在的位置, 然后用
write系统调用把arg2写入到对应扇区。
 */
void wsect(uint, void*);
```

```

/*
 * 作用： 依据inode的编号把它对应的数据结构写入到硬盘的
inode区
 * uint: inode的编号。
 * struct dinode*: 用来指向inode对应的数据结构
 * 原理：
 */
void winode(uint, struct dinode*);

/*
 * 作用：依据inode的编号获取它的数据结构
 * inum : inode的编号
 * *ip : 用来指向inode对应的数据结构
 * 原理：
 */
void rinode(uint inum, struct dinode *ip);

/*
 * 作用：从指定扇区读出数据
 * sec : sect No.
 * *buf : 缓冲区地址。
 * 原理 : 首先用lseek系统调用定位到扇区所在的位置, 然后
用read系统调用从对应扇区把数据读到缓冲区里。
 */
void rsect(uint sec, void *buf);

/*
 * 作用 : 分配一个inode
 * type : inode类型, 有目录、文件、设备三种类型
 * 返回值 : inode号
 * 原理 : 分配一个inode号并初始化一个dinode数据结构,
然后调用winode把这个数据结构写入到inode区里inode号对应
的位置。
 */
uint ialloc(ushort type);

```

```
/*
 * 作用   : 向inode里添加内容
 * inum   : inode号
 * *xp    : 要添加的信息的指针
 * n      : 要添加的信息的大小
 * 原理   : 首先得到这个文件的大小, 把它赋值给off变量。进而得到当前文件末尾的块编号fbn, 进而得到分配的块号x。然后以buf为缓冲区, 把不大于1个块的内容写入x号块。调整n, off, p的值, 并持续循环, 直到把所有的内容都写到块里。最后, 修改inode的属性, 并调用winode把inode的信息再写回磁盘。
 */
void iappend(uint inum, void *p, int n);
```

格式化的过程

本程序的功能是创建一个格式化的镜像。至少要有两个参数, 第一个参数是镜像名称, 第二个及以后的参数是要放到镜像里的文件的名称。镜像有1000个块, 每个块1KB, 故镜像大小为1M。

数据结构de代表了目录文件里的每一项。

字符数组buf的大小是一个块的大小, 这样可以方便地把buf里的数据写入到扇区里。

整数freeinode代表了空闲inode的编号。每分配出一个inode它就执行加1操作。

首先, 使用系统调用open, 以第一个参数为文件名创建一个文件, 其文件描述符赋值给fsfd。

然后, 计算各个数据区的大小。

- 启动块和超级块点两个块。
- 日志块nlog占30个块，inode块ninodeblocks占的块数（即inode的总数200除以每个块可以保存的inode数据结构的数量，其值为13），位图占用的块数nbitmap，其值为1。
- 如上的块组成元数据区nmeta，共占46个块。
- 除了元数据区，镜像空间剩下的即为数据区nblocks，共占954个块。

然后，填充超级块的数据结构。xint的作用是让数字满足内存里的表示形式，即低位放在低字节，高位放在高字节。

然后，在for循环里调用wsect函数把整个镜像填充为0。

然后，把超级块的内容写入到1号扇区。

然后，为根目录分配一个inode号rootino。

然后，为根目录添加条目.和..。

然后，在一个for循环里，把要传入的文件依次复制到镜像的根目录下。

然后，修改根目录所对应的inode的数据结构的size属性。

最后，执行balloc函数，把数据块的使用情况写入到位图区。